INTERVIEWHUB · INTERVIEW PREP

OpenStack Private Cloud — InterviewHub

Platform, SRE & DevOps interview guide

Questions	**150 questions · spoken answers included**
Online (always latest)	**interviewhub.eknathalabs.com/openstack_interview_guide.html**
Printed	**2026-10-21**

Free to use and share for personal study. Answers are written for interview practice, so check command flags and versions against the official documentation before running anything in production.

certs.eknathalabs.com

Q1 What is OpenStack and which core services make up a typical private cloud? Core

🎙 Spoken answer · ~58 s

OpenStack is an open-source cloud operating system that turns a pool of servers, storage and network gear into a self-service infrastructure-as-a-service platform. You consume it through APIs, a CLI and the Horizon dashboard. The core services I always name are Keystone for identity, Nova for compute, Neutron for networking, Glance for images, Cinder for block storage, Placement for resource inventory, and Horizon for the UI. Swift gives object storage, Heat does orchestration, Octavia handles load balancing, and Ironic manages bare metal. Each service exposes a REST API, keeps state in a shared database, and talks to its own agents over a message queue, usually RabbitMQ. In an interview I stress that OpenStack is a set of loosely coupled projects, so you pick the ones you need. A small cloud might run only Keystone, Nova, Neutron, Glance and Cinder.

Likely follow-ups

- Which services are optional?
- How do services authenticate to each other?

OpenStack is a set of loosely coupled services exposing REST APIs. The essentials are Keystone (identity), Nova (compute), Neutron (network), Glance (images), Cinder (block storage) and Placement (inventory), with Horizon as the UI.

```
openstack service list
```

```
openstack catalog list
```

```
openstack endpoint list
```

Interview tip: Say that services share a database and a message bus, then name the minimum viable set.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q2 Describe the OpenStack control-plane architecture: API, database, message queue and agents.

Core

🎙 **Spoken answer · ~58 s**

Every OpenStack service follows a similar shape. An API process receives REST calls and validates the token with Keystone. It writes the desired state to a MariaDB or MySQL database, usually a Galera cluster. For long-running work it casts or calls a message over RabbitMQ using oslo.messaging. Worker daemons, such as nova-conductor, nova-scheduler and nova-compute, or neutron agents, pick up those messages and do the real work on hosts. Results flow back through the queue or the database. This is why RabbitMQ and the database are the two shared dependencies you protect most carefully. If the queue backs up, API calls succeed but nothing happens. If the database is slow, everything slows down. Behind a load balancer such as HAProxy, you run several copies of every API and conductor, which gives horizontal scale and high availability on the controller nodes.

Likely follow-ups

- Why is nova-conductor needed?
- What breaks first when RabbitMQ is unhealthy?

API services validate tokens, persist state in Galera, and hand work to workers through RabbitMQ (oslo.messaging). The database and the queue are the shared dependencies to protect.

```
openstack compute service list
```

```
rabbitmqctl cluster_status
```

```
mysql -e 'SHOW STATUS LIKE "wsrep_cluster_size"'
```

Interview tip: Explain the failure mode: API accepts the request but no worker consumes it.

📚 **Official docs**

- OpenStack documentation
- Galera Cluster documentation

certs.eknathalabs.com

Q3 Walk through what happens, step by step, when a user runs openstack server create.

Core

🎙 Spoken answer · ~59 s

The client first gets a token from Keystone, then calls the Nova API with the flavor, image and network. Nova validates the request, checks quota, and writes an instance record in the database in the building state. The nova-scheduler asks the Placement service for candidate resource providers that have enough vCPU, RAM and disk, then applies filters and weights to choose a host. The conductor sends the build request to nova-compute on that host over the message queue. Compute asks Glance for the image, asks Neutron to create and bind a port, and asks Cinder if there is a volume. Libvirt then defines and starts the guest on KVM. Neutron wires the port with OVS or OVN and DHCP delivers the address. When the guest is running, Nova marks the instance ACTIVE. Knowing this order is how you localise a failure fast.

Likely follow-ups

- Where does the scheduler get its data?
- At which step does a no valid host error appear?

Keystone token, Nova API and quota, scheduler with Placement, conductor to compute, then Glance image, Neutron port and Cinder volume, and finally libvirt starts the guest.

```
openstack server create --flavor m1.small --image ubuntu-22.04 --network private --key-name mykey web1
```

```
openstack server show web1 -c status -c fault
```

```
openstack server event list web1
```

Interview tip: Narrate it in order, then say which log you would open at each step.

📚 Official docs

- OpenStack documentation
- Nova documentation

certs.eknathalabs.com

Q4 OpenStack versus VMware vSphere versus Kubernetes: how do you position each?

Core

🎙 Spoken answer · ~56 s

They solve different layers. VMware vSphere is a polished, commercially supported virtualisation stack with strong tooling and licensing cost. OpenStack is an open-source infrastructure API layer that gives you multi-tenant compute, network and storage as a service, with no per-socket licence but real operational effort. Kubernetes orchestrates containers and assumes it already has machines and networks underneath. They are often combined: Kubernetes runs on top of OpenStack VMs or bare metal, using Cinder volumes and Octavia load balancers. I choose OpenStack when an organisation wants a private cloud with self-service, tenant isolation, open APIs, and freedom from licence lock-in, and has a team to run it. I pick VMware when operational simplicity and vendor support outweigh cost. I pick plain Kubernetes when the workloads are all containers and there is no need to offer VMs.

Likely follow-ups

- When would you not run OpenStack?
- How does Kubernetes consume OpenStack?

vSphere is a supported virtualisation product, OpenStack is an open IaaS API layer, and Kubernetes orchestrates containers on top of either. They are complementary, not exclusive.

Interview tip: Frame it as layers, not rivals, and be honest about the operational cost of running OpenStack.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q5 Explain regions, availability zones, host aggregates and cells. How do they differ?

Core

🎙 **Spoken answer · ~62 s**

These are four different ways to partition a cloud. A region is a complete deployment with its own API endpoints, usually in a separate site, sharing only Keystone if you choose. An availability zone is a user-visible failure domain inside a region, such as a rack or power zone, which users select to spread instances. A host aggregate is an operator-only grouping of compute hosts with metadata, used for things like GPU hosts, SSD hosts or licence-bound hosts, and tied to flavors through scheduler filters. Cells are a Nova scaling mechanism: each cell has its own database and message queue for its computes, and a top-level API database routes to them. So AZs are for tenants, aggregates are for operators, cells are for scale, and regions are for geography. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- Can a host be in several aggregates?
- Why do cells reduce load on RabbitMQ?

Region: a whole deployment. AZ: tenant-visible failure domain. Aggregate: operator grouping with metadata. Cell: scaling shard with its own DB and queue.

```
openstack availability zone list
```

```
openstack aggregate create --zone az1 gpu-hosts
```

```
openstack aggregate set --property gpu=true gpu-hosts
```

Interview tip: Use the one-line summary: AZs for tenants, aggregates for operators, cells for scale.

📚 **Official docs**

- Nova: Cells
- Nova: Availability zones
- Nova: Host aggregates

certs.eknathalabs.com

Q6 What is the Placement service and why was it split out of Nova? Core

🎙 Spoken answer · ~57 s

Placement is a small service that tracks inventory and usage of resources. A resource provider, such as a compute node, reports inventories of VCPU, MEMORY_MB and DISK_GB. Consumers, such as instances, claim allocations against those providers. Placement also supports traits, which are qualitative tags like HW_CPU_X86_AVX2 or CUSTOM_GPU, and aggregates and nested providers for things like NUMA nodes and GPUs. Nova used to compute free capacity itself in the scheduler and that was slow and racy. Splitting it out gave a generic, fast, queryable inventory service that Nova, Cyborg and Neutron can all use, for example for bandwidth-aware scheduling. The scheduler now asks Placement for allocation candidates first, then applies filters. When capacity looks wrong, I compare the Placement allocations with what Nova thinks is running, because drift between them causes phantom no valid host errors.

Likely follow-ups

- What are traits used for?
- How do you fix stale allocations?

Placement stores resource providers, inventories, allocations and traits. Nova asks it for allocation candidates before running its filters, which is faster and less racy than the old in-scheduler accounting.

```
openstack resource provider list
```

```
openstack resource provider inventory list <uuid>
```

```
openstack resource provider usage show <uuid>
```

Interview tip: Mention allocation drift as the thing you actually debug in production.

📚 Official docs

- OpenStack Nova
- OpenStack documentation
- Placement documentation

certs.eknathalabs.com

Q7 How does the Nova scheduler pick a host? Explain filters and weighers.

Core

🎙 Spoken answer · ~64 s

Scheduling is a two-stage funnel. First, Placement returns allocation candidates that have enough raw resources. Second, the scheduler runs an ordered list of filters, each of which can reject a host. Common ones are AvailabilityZoneFilter, ComputeFilter, ImagePropertiesFilter, ServerGroupAntiAffinityFilter, NUMATopologyFilter and AggregateInstanceExtraSpecsFilter. Hosts that survive are then ranked by weighers, for example RAMWeigher, which by default spreads load onto the host with most free RAM, or a custom weigher that packs instead. The highest weight wins, with some randomness among the top hosts to avoid stampedes. When you see no valid host was found, the useful move is to read the scheduler log, which prints how many hosts each filter removed, and that tells you whether the issue is capacity, a trait, an aggregate or an affinity rule. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- Spread versus pack: how do you change it?
- How do you debug no valid host?

Placement narrows candidates by raw capacity, filters reject hosts on policy, and weighers rank the remainder. The scheduler log shows how many hosts each filter removed.

```
grep 'Filter' /var/log/nova/nova-scheduler.log | tail
```

```
openstack hypervisor stats show
```

```
openstack flavor show m1.small -c properties
```

Interview tip: Say you debug no valid host by reading per-filter counts in the scheduler log.

📚 Official docs

- OpenStack Nova
- Nova: Scheduling and filters
- OpenStack documentation

certs.eknathalabs.com

Q8 What are flavors and how do you design them well?

Core

🎙 Spoken answer · ~65 s

A flavor defines the size of an instance: vCPUs, RAM, root disk, ephemeral disk and swap, plus extra specs that control behaviour. Extra specs can pin CPUs, request hugepages, set NUMA topology, request a trait through the trait prefix, or select an aggregate. Good flavor design keeps a small, predictable catalogue rather than hundreds of custom sizes, because each one fragments capacity. I group them into families, general purpose, compute optimised, memory optimised, and GPU, with consistent ratios of RAM to vCPU. Root disk sizes are small if instances boot from volume. I make dedicated-CPU flavors public only where needed, and I use private flavors for special tenants. Flavors are immutable in practice, so changing one means creating a new flavor and resizing instances, which is why planning them up front matters. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- How do you pin CPUs through a flavor?
- Why prefer boot from volume with small root disks?

A flavor sets vCPU, RAM and disk, and extra specs add pinning, hugepages, traits and aggregate selection. Keep a small catalogue grouped by family to avoid capacity fragmentation.

```
openstack flavor create --vcpus 4 --ram 8192 --disk 20 m1.large
```

```
openstack flavor set m1.large --property hw:cpu_policy=dedicated
```

```
openstack flavor set m1.large --property trait:CUSTOM_GPU=required
```

Interview tip: A short, consistent flavor catalogue is a capacity-planning argument, not just a UX one.

📚 Official docs

- Nova: Flavors

certs.eknathalabs.com

Q9 Explain the lifecycle and common states of a Nova instance.

Core

🎙 Spoken answer · ~59 s

An instance starts in BUILD while the scheduler, compute, image and network work completes. It becomes ACTIVE when running, SHUTOFF when stopped, and PAUSED or SUSPENDED when the guest is frozen in memory or saved to disk. ERROR means a task failed and the fault field tells you why. Transitional states like RESIZE, VERIFY_RESIZE, MIGRATING, REBUILD and SHELVED_OFFLOADED show an operation in flight, and the separate task state field shows exactly which step. SHELVED_OFFLOADED releases the hypervisor resources and keeps only an image, which is a good way to save capacity for idle servers. DELETED is final, with soft delete optional. In practice I always look at three fields together: status, task_state and power_state, because an instance can be ACTIVE in the database while the guest is actually shut off on the host, and that mismatch is a classic troubleshooting clue.

Likely follow-ups

- What does VERIFY_RESIZE mean?
- How do you reset a stuck instance state?

BUILD, ACTIVE, SHUTOFF, PAUSED, SUSPENDED, SHELVED_OFFLOADED, ERROR and DELETED. Check status, task_state and power_state together; a mismatch points at a host-side problem.

```
openstack server show web1 -c status -c OS-EXT-STS:task_state -c OS-EXT-STS:power_state
```

```
openstack server set --state active web1
```

```
openstack server shelve web1
```

Interview tip: Mention power_state versus status mismatch: it signals the database and the hypervisor disagree.

📚 Official docs

- OpenStack Nova
- OpenStack documentation

certs.eknathalabs.com

Q10 How does cloud-init and the metadata service work in OpenStack?

Core

🎙 **Spoken answer · ~60 s**

When an instance boots, cloud-init inside the image asks for configuration at the well-known link-local address 169.254.169.254. Neutron makes this reachable through a metadata proxy, either in the router or DHCP namespace, or the OVN metadata agent, which forwards the request to the Nova metadata API and adds headers that identify the instance. Nova returns the hostname, SSH public key, user data and network details. Alternatively, a config drive can be attached as a small read-only disk containing the same data, which works even when there is no network or DHCP. User data is where you inject scripts or cloud-config YAML for first-boot setup. If an instance has no SSH key or user data did not run, I check whether it reached the metadata endpoint from inside the guest, then the metadata agent logs, then whether a config drive would be more reliable.

Likely follow-ups

- When would you prefer a config drive?
- What breaks if the metadata agent is down?

cloud-init queries 169.254.169.254, which Neutron proxies to the Nova metadata API. A config drive is the no-network alternative. User data is the first-boot hook.

```
curl http://169.254.169.254/openstack/latest/meta_data.json
```

```
openstack server create --config-drive true ...
```

```
journalctl -u cloud-init
```

Interview tip: Name both paths: the network metadata service and the config drive.

📚 **Official docs**

- Nova: Metadata service
- OpenStack documentation

certs.eknathalabs.com

Q11 How are projects, quotas and limits managed, and what happens when a quota is hit? Core

🎙 **Spoken answer · ~56 s**

Every resource belongs to a project, formerly called a tenant, and each project has quotas per service: instances, cores, RAM, volumes, gigabytes, floating IPs, security groups and so on. When a user asks for a resource, the service checks current usage against the quota and rejects the request with a quota exceeded error if it would be crossed. Operators set defaults and per-project overrides, and Nova and Cinder can use unified limits through Keystone for a consistent model. Quotas protect shared capacity and prevent one noisy tenant from consuming everything, but they are not guarantees of capacity. They do not reserve hardware. So I pair quotas with capacity monitoring, and I audit usage sync, because after failures the recorded usage can drift from reality and I may need to recount it for a project.

Likely follow-ups

- Does a quota reserve capacity?
- How do you fix quota usage drift?

Quotas cap per-project consumption and are enforced at request time. They limit but do not reserve capacity, so combine them with real capacity monitoring.

```
openstack quota show <project>
```

```
openstack quota set --cores 200 --ram 512000 <project>
```

```
openstack limits show --absolute
```

Interview tip: Be explicit that quota is a ceiling, not a reservation.

📚 **Official docs**

- Nova: Quotas

certs.eknathalabs.com

Q12 What is the service catalog and how do endpoints work?

Core

🎙 Spoken answer · ~67 s

The service catalog is the list Keystone returns inside a token response, telling clients where every service lives. Each service, such as compute or network, has endpoints for three interfaces: public, internal and admin, and for each region. Clients use the public endpoint, services talking to each other inside the cloud use the internal endpoint, and administrative operations may use the admin endpoint. This split lets you put public APIs behind a TLS-terminating load balancer with a public name while internal traffic stays on a private network. A common outage cause is a wrong endpoint URL, for example pointing to an old controller IP after a migration, which makes only some services fail. I verify with the catalog, confirm each URL responds, and make sure the region name matches between clients and services. I would finish by naming the one log line or metric that proves the change worked, because verification is what separates a confident answer from a guess.

Likely follow-ups

- Why separate internal and public endpoints?
- What breaks when an endpoint URL is stale?

Keystone returns a catalog of services with public, internal and admin endpoints per region. Clients pick by interface and region, so a stale URL breaks only the services that use it.

```
openstack catalog show compute
```

```
openstack endpoint list --service compute
```

```
openstack --os-interface internal server list
```

Interview tip: Mention interfaces and region together, since mismatches are the usual bug.

📚 Official docs

- Keystone documentation

certs.eknathalabs.com

Q13 How do OpenStack microversions and API versioning work?

Core

🎙 Spoken answer · ~62 s

Most OpenStack APIs keep a stable major version, for example Compute v2.1, and add features through microversions. A client states the microversion it wants with a header such as X-OpenStack-Nova-API-Version, or the CLI flag os-compute-api-version. The server then behaves as defined for that microversion, and each new minor feature bumps the number. If the client asks for a version higher than the server supports, it gets a clear error, and if it omits the header it receives the baseline behaviour, which keeps old scripts working. The discovery document at the API root lists the minimum and maximum supported versions. For automation I pin an explicit microversion so behaviour does not change under me during upgrades, and I test upgrades by running the same scripts at the old and new maximum. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- Why pin a microversion in scripts?
- How do you find the maximum supported version?

Stable major versions, with microversions adding behaviour. Clients opt in through a header; omitting it gives baseline behaviour. Pin the version in automation.

```
openstack versions show --service compute
```

```
openstack --os-compute-api-version 2.90 server list
```

```
curl -s $NOVA_ENDPOINT | jq .
```

Interview tip: Explain that pinning keeps automation stable across upgrades.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q14 What is oslo and why do operators care about it?

Core

🎙 Spoken answer · ~64 s

Oslo is the set of shared Python libraries used by every OpenStack project, so each service does not reinvent the basics. The ones operators meet most are oslo.messaging for RPC over RabbitMQ, oslo.db for database access, oslo.config for configuration files, oslo.log for logging, oslo.policy for authorisation rules, and oslo.cache for caching. Because these are shared, behaviour is consistent: the same config option style, the same log format, the same retry semantics. It also means tuning oslo.messaging, for instance heartbeat and timeout settings, affects all services at once, and a bug there is cloud-wide. When I troubleshoot, I often start with oslo.messaging errors, because a message timeout in one service is usually a symptom of a broken queue, a dead consumer, or clock and network problems rather than a bug in that service itself. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- Which oslo option would you tune for RabbitMQ flaps?
- How does oslo.policy relate to RBAC?

Oslo is the shared library layer: messaging, db, config, log, policy and cache. Tuning or bugs there affect every service, which is why messaging errors are the first thing to check.

```
grep -i 'MessagingTimeout' /var/log/nova/*.log
```

```
crudini --get /etc/nova/nova.conf oslo_messaging_rabbit
```

```
oslopolicy-checker --help
```

Interview tip: Say you read MessagingTimeout errors as a queue or consumer symptom, not a service bug.

📚 Official docs

- oslo.messaging documentation

certs.eknathalabs.com

Q15 How do you read and manage OpenStack configuration and logs effectively? Core

🎙 Spoken answer · ~57 s

Each service reads an ini-style file such as nova.conf or neutron.conf, plus drop-in directories in newer releases, and some changes only apply after a restart while others support a mutable reload through SIGHUP. In containerised deployments like Kolla, the configuration is rendered by Ansible and mounted into containers, so I change it in the deployment tooling, never by hand on the host, otherwise the next run overwrites it. For logs, every service writes to its own file or to journald, and a request ID, req-uuid, is propagated across services so I can follow one API call from Nova to Neutron to Cinder. I ship logs to a central system, index by request ID and instance UUID, and keep debug off by default, switching it on for one service temporarily. Good logging discipline is half of OpenStack operations.

Likely follow-ups

- What is a request ID used for?
- How do you enable debug temporarily?

ini files per service, reloaded by restart or SIGHUP. In containers, change configuration through the deployment tool. Follow one request across services with its req-id.

```
grep req-<uuid> /var/log/*/*.log
```

```
docker exec nova_api cat /etc/nova/nova.conf
```

```
kolla-ansible reconfigure -t nova
```

Interview tip: The request ID across services is the single best debugging tool; mention it.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q16 What does the release cycle look like and how do SLURP releases affect upgrades?

Core

🎙 **Spoken answer · ~62 s**

OpenStack ships a new release roughly every six months, named alphabetically, for example 2024.1 Caracal and 2024.2 Dalmatian. Each release is supported by the community for a limited maintenance window, with extended maintenance after that. Since 2023.1, the project introduced SLURP, Skip Level Upgrade Release Process, where every other release is designated a SLURP release, and you can upgrade directly from one SLURP release to the next, skipping the one between. That halves the number of upgrade events to roughly one per year while still allowing the intermediate non-SLURP releases for those who want newer features. For planning, I pick a SLURP path, test the upgrade in a staging cloud that mirrors production, read the release notes for removed options and deprecations, and schedule it as a rolling upgrade. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- What is the difference between SLURP and non-SLURP?
- How do you test an upgrade safely?

Two releases a year; every other one is a SLURP release you can upgrade between directly, so one planned upgrade per year is possible.

```
openstack --version
```

```
grep -i 'upgrade' release-notes.txt
```

```
kolla-ansible upgrade -i inventory
```

Interview tip: Mention you test the exact upgrade path in staging and read the removed-options notes.

📚 **Official docs**

- Nova: Upgrades

certs.eknathalabs.com

Q17 How do you check the health of a running OpenStack cloud quickly?

Core

🎙 Spoken answer · ~65 s

I check in layers. First the control-plane services: openstack compute service list and network agent list show whether every nova-compute, scheduler, conductor and neutron agent is up. Second, the shared dependencies, which are Galera cluster size and sync state, RabbitMQ cluster status and queue depth, and the HAProxy backends. Third, a real functional probe: create a tiny instance, attach a floating IP, ping it and delete it, which exercises Keystone, Nova, Glance, Neutron and the data plane together. I automate that as a canary every few minutes. Fourth, resource headroom, meaning hypervisor usage, Ceph capacity and placement allocations. Fifth, logs and alerts for errors and stuck instances. A green service list with a failing canary tells you the problem is in the integration or data plane, not in a daemon. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- What would your canary test include?
- Which metric tells you RabbitMQ is struggling?

Check service and agent lists, Galera and RabbitMQ state, run a synthetic canary that boots a VM and pings it, then check capacity headroom.

```
openstack compute service list
```

```
openstack network agent list
```

```
openstack volume service list
```

Interview tip: A synthetic end-to-end canary catches what a green service list misses.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q18 What is Horizon and when do you prefer the CLI, SDK or API instead?

Core

🎙 Spoken answer · ~65 s

Horizon is the Django web dashboard that lets users and admins manage projects, instances, networks and volumes, and it just calls the public APIs, so anything it does you can also do elsewhere. I use it for quick exploration and for users who are new to the cloud. For daily operations and automation I prefer the unified openstack CLI because it is scriptable and consistent across services, and for programmatic work I use openstacksdk in Python or the Terraform and Ansible providers, which give repeatable, reviewable infrastructure. Raw REST calls are mostly for debugging when I want to see headers and microversions. A key operational point is authentication: clouds.yaml keeps credentials and endpoints for multiple clouds and projects, so the same tooling works everywhere without sourcing environment files by hand. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- What is clouds.yaml?
- Why is infrastructure as code better than Horizon clicks?

Horizon is a UI over the same APIs. Use the openstack CLI for scripting, openstacksdk, Terraform or Ansible for repeatable automation, and clouds.yaml for credentials.

```
openstack --os-cloud prod server list
```

```
python -c 'import openstack; print(openstack.connect(cloud="prod").compute.servers())'
```

```
cat ~/.config/openstack/clouds.yaml
```

Interview tip: Say that anything repeatable belongs in code, not in dashboard clicks.

📚 Official docs

- OpenStack documentation
- openstacksdk documentation

certs.eknathalabs.com

Q19 How do images work in Glance and what formats do you use?

Core

🎙 Spoken answer · ~61 s

Glance stores and serves virtual machine images and their metadata. An image has a disk format, such as qcow2, raw, vmdk or iso, and a container format, usually bare. With a Ceph backend, I convert images to raw so that instances can be created as copy-on-write clones of the image in RBD, which makes boot take seconds instead of copying gigabytes. Images have visibility settings, private, shared, community or public, and properties like hw_disk_bus, hw_vif_model and os_distro that tell Nova how to present hardware to the guest. I keep golden images small, patched regularly, with cloud-init installed and no secrets inside, and I sign them where possible. Glance also supports multiple backends and image caching on compute hosts for speed. I would finish by naming the one log line or metric that proves the change worked, because verification is what separates a confident answer from a guess.

Likely follow-ups

- Why raw images on Ceph?
- How do you control who can see an image?

Glance stores images with format and metadata properties. On Ceph, use raw so boots are instant copy-on-write clones. Keep golden images small, patched and secret-free.

```
openstack image create --disk-format raw --container-format bare --file ubuntu.raw --public ubuntu-22.04
```

```
qemu-img convert -f qcow2 -O raw ubuntu.qcow2 ubuntu.raw
```

```
openstack image set --property hw_disk_bus=scsi ubuntu-22.04
```

Interview tip: The raw-on-Ceph point shows you understand the storage and compute interaction.

📚 Official docs

- OpenStack documentation
- Glance documentation

certs.eknathalabs.com

Q20 Explain live migration: how it works, its requirements and common failures.

Nova

🎙 Spoken answer · ~62 s

Live migration moves a running instance between compute hosts with minimal downtime. Libvirt copies the guest memory pages iteratively to the destination while the guest keeps running, then briefly pauses it, transfers the last dirty pages and the CPU state, and resumes it on the target. With shared storage such as Ceph, only memory moves. Without shared storage, block migration also copies the disk, which is much slower. Requirements are compatible CPU models between hosts, reachable libvirt and migration ports, consistent networking, matching image and volume access, and enough destination capacity. Common failures include CPU feature mismatches, a guest dirtying memory faster than it can be copied so it never converges, firewalls blocking ports, and mismatched Neutron port binding. Tools to help include auto-converge, post-copy and a configurable downtime limit. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- How do you handle a guest that never converges?
- What CPU mode do you set for migration-friendly clouds?

Libvirt pre-copies memory while the guest runs, then pauses briefly to finish and resume on the target. Shared storage keeps it quick; CPU model mismatches and high dirty rates are common failures.

```
openstack server migrate --live-migration web1
```

```
nova live-migration-force-complete web1 <migration-id>
```

```
openstack server migration list --server web1
```

Interview tip: Name auto-converge and post-copy as the answer for busy guests.

📚 Official docs

- OpenStack Nova
- Nova: Live migration

certs.eknathalabs.com

Q21 What is the difference between cold migration, resize, evacuate and rebuild?

Nova

🎙 Spoken answer · ~56 s

They look similar but serve different situations. Cold migration shuts the instance down, moves it to another host and starts it again, and is used for maintenance when live migration is not possible. Resize changes the flavor, and because the new size may not fit the old host, it also moves the instance; the state VERIFY_RESIZE lets the user confirm or revert. Evacuate is for failure: when a compute host is dead, you rebuild its instances on another host, keeping the same UUID, network ports and volumes, and for shared-storage setups the disk survives. Rebuild reinstalls an instance from a new or same image on the same host, wiping the root disk. Picking the right one is a classic scenario question, with the key words being planned versus failed host, and keep data versus reinstall.

Likely follow-ups

- What data is lost on rebuild?
- What must be true before you evacuate?

Cold migrate: planned move with downtime. Resize: new flavor, may move. Evacuate: recover from a dead host. Rebuild: reinstall the root disk from an image.

```
openstack server resize --flavor m1.large web1
```

```
openstack server resize confirm web1
```

```
nova evacuate web1
```

Interview tip: Frame it as planned versus failed host, and data kept versus wiped.

📚 Official docs

- OpenStack Architecture Design Guide
- Nova: Live migration
- Nova: Evacuate

certs.eknathalabs.com

Q22 What is CPU pinning and NUMA awareness, and when do you need them?

Nova

🎙 Spoken answer · ~62 s

Modern servers have multiple NUMA nodes, where memory and PCI devices are closer to certain cores. A guest that spans nodes pays a latency penalty for remote memory. With the flavor property hw:cpu_policy set to dedicated, Nova pins each guest vCPU to a specific host physical core, and with hw:numa_nodes it keeps the guest on specified NUMA nodes, aligning memory and vCPUs. Combined with hugepages, this gives predictable low latency and is standard for telco, NFV, databases and high-performance workloads. The trade-off is that pinned instances cannot be overcommitted and reduce scheduling flexibility, so I place them in a dedicated host aggregate, reserve host cores for the OS using cpu_dedicated_set and cpu_shared_set, and use separate flavors. For ordinary web workloads, shared CPUs with overcommit are the better cost choice. If the interviewer pushes further, I would explain how I would test this in a staging environment before touching production.

Likely follow-ups

- Why separate dedicated and shared CPU sets?
- What does hw:cpu_thread_policy do?

hw:cpu_policy=dedicated pins vCPUs to cores and NUMA settings keep memory local. Great for latency-sensitive loads, but no overcommit, so isolate with aggregates.

```
openstack flavor set rt --property hw:cpu_policy=dedicated --property hw:numa_nodes=1
```

```
crudini --set /etc/nova/nova.conf compute cpu_dedicated_set 2-23
```

```
lscpu | grep NUMA
```

Interview tip: State the trade-off: performance and predictability versus density.

📚 Official docs

- Nova: CPU topologies and pinning

certs.eknathalabs.com

Q23 How do hugepages help, and how are they configured for instances?

Nova

🎙 Spoken answer · ~63 s

Normal memory uses 4 KB pages, so large guests need many page-table entries, and the CPU translation cache, the TLB, misses often. Hugepages, typically 2 MB or 1 GB, cut that overhead and improve memory-intensive and packet-processing workloads, and they are required for DPDK. On the host you reserve hugepages at boot with kernel parameters, so they are not fragmented, and you tell Nova how much memory is for hugepages. On the instance side you set the flavor property hw:mem_page_size to large, 2MB or 1GB. Hugepage memory is not swappable and not shared with normal allocations, so you must size reservations carefully; too many starves normal guests and the host, too few causes scheduling failures. I also remember that guests with hugepages cannot use memory overcommit. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- Why are hugepages needed for DPDK?
- What happens if hugepages are exhausted?

Hugepages reduce TLB misses for big or packet-heavy guests. Reserve them at host boot, set hw:mem_page_size on the flavor, and accept no memory overcommit.

```
grep Huge /proc/meminfo
```

```
openstack flavor set dpdk --property hw:mem_page_size=1GB
```

```
grubby --update-kernel=ALL --args='hugepagesz=1G hugepages=64'
```

Interview tip: Link hugepages to DPDK and NFV to show you know the use case.

📚 Official docs

- Nova: Huge pages
- Nova documentation

certs.eknathalabs.com

Q24 How do CPU and RAM overcommit ratios work and how do you choose them?

Nova

🎙 Spoken answer · ~63 s

Overcommit lets the scheduler allocate more virtual resources than physical ones, because most guests are idle most of the time. The cpu_allocation_ratio and ram_allocation_ratio are set per compute node, or through Placement inventory, with typical defaults of 16 for CPU and 1.5 for RAM. CPU overcommit is relatively safe because the kernel scheduler time-slices, but it hurts latency under contention; I watch steal time and load. RAM overcommit is riskier, because exhausting host memory triggers swapping or the OOM killer, so I use a low ratio like 1.0 to 1.2 for production. Disk can be overcommitted with thin provisioning but needs monitoring. I choose ratios by workload class, conservative for databases and latency-sensitive tenants, higher for dev and test aggregates, and I confirm with real utilisation data, not guesses. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- Which metric shows CPU contention?
- Where do you set the ratio so it is respected by Placement?

Allocation ratios let scheduling exceed physical capacity. CPU 4 to 16 is common, RAM close to 1. Set per aggregate by workload and watch steal time and memory pressure.

```
crudini --set /etc/nova/nova.conf DEFAULT cpu_allocation_ratio 8.0
```

```
openstack resource provider inventory show <uuid> VCPU
```

```
sar -u 1 5   # check %steal
```

Interview tip: Say conservative for databases, aggressive for dev, and validate with steal time.

📚 Official docs

- Linux kernel docs: /proc/sys/vm
- Nova: Scheduling and filters

certs.eknathalabs.com

Q25 What are Nova cells v2 and why would you deploy them?

Nova

🎙 Spoken answer · ~65 s

Cells v2 is Nova's horizontal scaling design. The top level has the API database, which holds instance mappings and host mappings, and the API services. Below that, each cell has its own database and message queue, serving a group of compute nodes. A request is routed to a cell by the scheduler and conductor, and the instance data lives in that cell. This bounds the size of any one RabbitMQ cluster and database, which are the usual bottlenecks at a few thousand compute nodes, and it gives a failure domain, since a broken cell affects only its hosts. There is always at least cell0, which holds instances that failed to schedule, and a default cell1. I would introduce extra cells when node counts, message rates or upgrade blast radius justify the added complexity. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- What is cell0 for?
- Which services run per cell versus globally?

Cells split compute nodes into groups with their own DB and queue under a global API database. They bound RabbitMQ and DB load and limit failure blast radius.

```
nova-manage cell_v2 list_cells
```

```
nova-manage cell_v2 discover_hosts --verbose
```

```
openstack hypervisor list
```

Interview tip: Cells are about bounding the message bus and DB, not about regions.

📚 Official docs

- OpenStack Nova
- Nova: Cells
- OpenStack documentation

certs.eknathalabs.com

Q26 A compute node dies. What is your recovery procedure?

Nova

🎙 Spoken answer · ~58 s

First confirm it is really down, not just a service issue, by checking the host, the hypervisor in nova and the console or BMC. If it is dead, I fence it so it cannot come back and corrupt shared disks, which is critical with shared storage. I mark the service forced down so Nova stops waiting for it, then evacuate the instances to healthy hosts, either one by one or in bulk with the host-evacuate approach. Instances on shared storage such as Ceph keep their disk and boot again with the same identity. Instances on local ephemeral disk lose data, so I tell the owners. I then check that ports, volumes and floating IPs are re-bound, repair or replace the hardware, and re-enable the service. Afterwards I review why detection and recovery were not automatic and add monitoring or Masakari.

Likely follow-ups

- Why fence before evacuate?
- What is Masakari?

Verify the failure, fence the host, force the service down, evacuate instances (shared storage preserves disks), verify networking, then repair and re-enable. Automate with Masakari.

```
openstack compute service set --down <host> nova-compute
```

```
nova host-evacuate <host>
```

```
openstack server list --host <host> --all-projects
```

Interview tip: Always say fencing comes before evacuation; that is the data-safety point.

📚 Official docs

- OpenStack Nova
- Nova: Evacuate

certs.eknathalabs.com

Q27 How do server groups and affinity rules work?

Nova

🎙 Spoken answer · ~57 s

A server group is a policy container that you attach to instances at boot using a scheduler hint. The policies are affinity, which places all members on the same host, anti-affinity, which forces them onto different hosts, and the soft variants that try but do not fail the build. The common use is high availability: put the replicas of a database or a stateless tier into an anti-affinity group so one host failure cannot take them all out. The limitation is that anti-affinity needs enough hosts, so a strict group of five on a three-host cloud will fail with no valid host. Also, the rule is enforced at build and during live migration and evacuate, which can block maintenance. I use soft anti-affinity when capacity is tight, and strict where the application truly cannot survive co-location.

Likely follow-ups

- What is the difference between anti-affinity and soft anti-affinity?
- How does anti-affinity affect maintenance?

Server groups with anti-affinity spread replicas across hosts and affinity co-locates them. Strict policies fail when hosts run out, so use the soft variant when capacity is tight.

```
openstack server group create --policy anti-affinity db-group
```

```
openstack server create --hint group=<group-id> --flavor m1.large --image ubuntu db1
```

```
openstack server group show db-group
```

Interview tip: Mention that strict anti-affinity can block live migration during maintenance.

📚 Start here (official docs)

- OpenStack documentation

certs.eknathalabs.com

Q28 How do you provide GPUs to instances: passthrough versus vGPU?

Nova

🎙 Spoken answer · ~56 s

There are two main approaches. With PCI passthrough, a whole physical GPU is assigned to one instance through VFIO, giving near native performance but no sharing. You enable IOMMU, whitelist the device in nova.conf, create a flavor with a pci alias, and Nova schedules onto hosts with a free device. With vGPU, an NVIDIA or similar driver splits one card into mdev types, and Nova assigns those virtual devices to several instances, which raises utilisation and suits VDI and smaller inference jobs, though it needs vendor licences and has migration limits. I choose passthrough for heavy training and full-card workloads, and vGPU for shared, smaller workloads. Either way I put GPU hosts in a dedicated aggregate, tag them with traits so the scheduler is precise, and monitor temperature, memory and utilisation with the vendor exporter.

Likely follow-ups

- What is needed on the host for passthrough?
- What are the migration limits of vGPU?

Passthrough gives a whole GPU to one guest with full performance. vGPU shares a card across guests. Use aggregates and traits to schedule precisely.

```
lspci -nn | grep -i nvidia
```

```
crudini --set /etc/nova/nova.conf pci device_spec
'{"vendor_id":"10de","product_id":"2204"}'
```

```
openstack flavor set gpu --property pci_passthrough:alias=a100:1
```

Interview tip: State the trade-off: dedicated performance versus utilisation.

📚 Official docs

- Kubernetes: Scheduling GPUs
- Nova: PCI passthrough
- Nova: Virtual GPUs

certs.eknathalabs.com

Q29 An instance is stuck in a transitional state such as BUILD or deleting. How do you fix it? Nova

🎙 **Spoken answer · ~58 s**

First I find out why, not just reset it. I look at the instance fault and task_state, then the nova-compute log on the host, using the request ID, to see what step hung: maybe image download, a Neutron port that never became active, a Cinder attach, or a libvirt error. If the underlying operation is gone, for example the compute service restarted mid-task, the database still shows the old task state, and the safe cleanup is to reset the state with openstack server set --state error, then delete or rebuild it. For a stuck delete I check for leftover ports, volumes and libvirt domains, which often block cleanup. I avoid editing the database directly except as a documented last resort. Afterwards I add a periodic job or alert for instances in the same state over a time threshold.

Likely follow-ups

- What is the safe way to reset an instance?
- How do you detect stuck instances proactively?

Read fault, task_state and the compute log by request ID to learn what hung. Then reset state to error and delete or rebuild; clean up ports and volumes. Alert on long-lived transitional states.

```
openstack server show web1 -c fault -c OS-EXT-STS:task_state
```

```
openstack server set --state error web1
```

```
openstack server delete --force web1
```

Interview tip: Diagnose before resetting; resetting hides the cause.

📚 **Official docs**

- Nova documentation

certs.eknathalabs.com

Q30 What hypervisors does Nova support and why is KVM the default choice?

Nova

🎙 Spoken answer · ~63 s

Nova supports several drivers through a compute driver interface. KVM through libvirt is by far the most used, and it is the best-tested path with all features such as live migration, NUMA, hugepages, vGPU and SR-IOV. Others include QEMU in pure emulation for testing, VMware vCenter, where Nova talks to vSphere, Hyper-V, and Ironic for bare metal instead of virtual machines. LXC and Virtuozzo drivers exist but see far less use. I choose KVM because it is open source, integrated in the Linux kernel, supported by every major distribution, performs close to bare metal with hardware virtualisation, and has the strongest community and vendor support. The decision to use VMware as a Nova backend usually comes from an existing investment, but then you lose several OpenStack-native features. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- When would you use the VMware driver?
- What does Ironic add?

KVM via libvirt is the mainstream, best-tested driver. Others exist, such as VMware, Hyper-V and Ironic for bare metal, but KVM has the richest feature support.

```
virsh list --all
```

```
egrep -c '(vmx|svm)' /proc/cpuinfo
```

```
openstack hypervisor list --long
```

Interview tip: Answer with KVM and give one reason beyond cost, such as feature coverage.

📚 Official docs

- OpenStack Nova
- OpenStack documentation
- libvirt documentation

certs.eknathalabs.com

Q31 How do you perform maintenance on a compute node without user impact?

Nova

🎙 Spoken answer · ~65 s

I plan it as a controlled drain. First I disable the compute service so the scheduler stops placing new instances there, with a reason recorded. Then I live-migrate instances away, in batches, checking that each migration completes and the guest remains reachable. Instances that cannot be live-migrated, such as those with pinned devices, get coordinated downtime or a cold migration with the owner informed. Once empty, I perform the work, such as a kernel patch, firmware update or hardware fix, then reboot, verify libvirt, nova-compute and the Neutron agent are healthy, and re-enable the service. For large fleets I use host aggregates and automation with Ansible, and I respect anti-affinity groups and capacity so the drain does not overload remaining hosts. Communication and a rollback plan complete the procedure. I would finish by naming the one log line or metric that proves the change worked, because verification is what separates a confident answer from a guess.

Likely follow-ups

- What can block a drain?
- How do you automate the loop for many hosts?

Disable the service, live-migrate instances in batches, handle non-migratable guests with coordination, do the work, verify agents, re-enable. Watch capacity and anti-affinity.

```
openstack compute service set --disable --disable-reason 'kernel patch' <host> nova-compute
```

```
openstack server list --host <host> --all-projects -f value -c ID | xargs -n1 openstack
server migrate --live-migration
```

```
openstack compute service set --enable <host> nova-compute
```

Interview tip: Say disable first, drain, work, verify, enable, in that order.

📚 Official docs

- OpenStack Nova

certs.eknathalabs.com

Q32 Explain the Neutron architecture: server, ML2 plugin, agents and mechanism drivers.

Neutron

🎙 Spoken answer · ~56 s

Neutron is the networking service. The neutron-server exposes the API and stores networks, subnets, ports and routers in the database. It uses the ML2 plugin, a modular framework with type drivers, which define how a network is built, such as flat, vlan, vxlan or geneve, and mechanism drivers, which actually program the back end, such as Open vSwitch, Linux bridge or OVN. In the classic OVS design, agents run on every node: the OVS agent wires ports, the DHCP agent runs dnsmasq per network, the L3 agent runs routers in namespaces, and the metadata agent proxies instance metadata. In the OVN design most of that logic moves into the OVN databases and ovn-controller on each host. I explain it as control plane in neutron-server and a data plane implemented by whichever mechanism driver you choose.

Likely follow-ups

- What does a type driver do versus a mechanism driver?
- Which agents disappear with OVN?

neutron-server holds the API and DB; ML2 combines type drivers (flat, vlan, vxlan, geneve) with mechanism drivers (OVS, OVN, Linux bridge) that program the data plane; agents run per node.

```
openstack network agent list
```

```
openstack extension list --network
```

```
grep mechanism_drivers /etc/neutron/plugins/ml2/ml2_conf.ini
```

Interview tip: Separate control plane (server and DB) from data plane (driver) in the answer.

📚 Official docs

- OpenStack Neutron
- Neutron: OVN driver
- OpenStack documentation

certs.eknathalabs.com

Q33 OVS versus OVN: what changed and why do most new clouds choose OVN?

Neutron

🎙 **Spoken answer · ~63 s**

ML2 with Open vSwitch uses a set of Python agents on every node: the OVS agent, L3 agent, DHCP agent and metadata agent, communicating over RabbitMQ, and routers run as Linux namespaces, which adds control-plane chatter and scale limits. OVN, Open Virtual Network, moves logical networking into a dedicated database model. Neutron writes logical switches, routers, ACLs and ports to the OVN northbound database, ovn-northd translates them into logical flows in the southbound database, and ovn-controller on each hypervisor programs OVS locally. Routing and security groups become OpenFlow rules distributed on every host, so DVR style east-west routing is built in, DHCP and metadata are handled by ovn-controller, and RabbitMQ load drops. The benefits are better scale, simpler operations and fewer moving parts, which is why new clouds default to OVN. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- What are the northbound and southbound databases?
- How do you migrate from OVS to OVN?

OVN replaces the agent-based design with logical networking in OVN databases, ovn-northd translation and ovn-controller on each host. Fewer agents, less RabbitMQ chatter, distributed routing by default.

```
ovn-nbctl show
```

```
ovn-sbctl show
```

```
ovs-vsctl show
```

Interview tip: Name northd and ovn-controller to show you know the pipeline.

📚 **Official docs**

- OpenStack Neutron
- Neutron: OVN driver

certs.eknathalabs.com

Q34 What is the difference between provider networks and tenant (self-service) networks?

Neutron

🎙 Spoken answer · ~56 s

Provider networks map directly onto physical networks that the operator defined, typically flat or VLAN, and instances attach to them like machines on the data-centre network. They are created by admins, have real routable addresses, and use the external router, so performance is simple and predictable, and they are common for production and for telco. Tenant networks, also called self-service, are created by users inside their project, usually as VXLAN or Geneve overlays, and are connected to external networks by a virtual router with SNAT and floating IPs. They give isolation and self-service, overlapping address ranges between tenants, and no involvement from the network team, at the cost of encapsulation overhead and a more complex data path. Many clouds use both: tenant networks for general workloads and provider networks for workloads that need direct addressing.

Likely follow-ups

- Why can tenants overlap IP ranges?
- When would you offer provider networks to tenants?

Provider networks are operator-defined mappings to physical VLANs with routable addresses. Tenant networks are user-created overlays routed to the outside through a virtual router and floating IPs.

```
openstack network create --provider-network-type vlan --provider-physical-network physnet1
--provider-segment 120 prov120
```

```
openstack network create tenant-net
```

```
openstack router create r1
```

Interview tip: Give a use case for each: provider for direct addressing, tenant for self-service isolation.

📚 Official docs

- Neutron: Provider networks
- Neutron: Self-service networks

certs.eknathalabs.com

Q35 How do VXLAN and Geneve overlays work, and what is the MTU consequence?

Neutron

🎙 Spoken answer · ~56 s

An overlay wraps each guest Ethernet frame inside a UDP packet that travels over the underlay between hypervisors. VXLAN uses a 24-bit virtual network identifier, so you can have about sixteen million segments instead of 4094 VLANs, with UDP port 4789. Geneve is the more extensible successor used by OVN, with variable-length options that carry metadata such as logical port identifiers. The cost is overhead: VXLAN adds about 50 bytes, Geneve about 58 or more, so if the underlay MTU is 1500 the guest MTU must be lowered to about 1442, otherwise large packets are fragmented or dropped, and you see the classic symptom of small requests working while large transfers hang. The fix is either a larger underlay MTU, such as 9000 jumbo frames, or advertising the correct smaller MTU to instances through DHCP.

Likely follow-ups

- How would you detect an MTU mismatch?
- Why jumbo frames on the underlay?

Overlays encapsulate guest frames in UDP between hypervisors, adding 50 to 58 bytes of header. Raise the underlay MTU or lower the guest MTU, or large packets will hang.

```
ping -M do -s 1400 <dst>
```

```
ip link show | grep mtu
```

```
openstack network show tenant-net -c mtu
```

Interview tip: The symptom-based answer: small packets work, big ones hang.

📚 Official docs

- RFC 1191: Path MTU discovery
- Cloudflare Learning: What is MTU?
- RFC 7348: VXLAN

alerts.eknathalabs.com

Q36 What is DVR and when is it useful with OVS ML2?

Neutron

🎙 Spoken answer · ~64 s

Distributed Virtual Routing moves the L3 function from a few central network nodes onto every compute node. Without it, all routed traffic and all floating IP traffic hairpin through the network node, creating a bottleneck and single point of failure. With DVR, east-west traffic between subnets is routed locally on the compute node, and north-south traffic for instances with floating IPs goes directly out from the compute node through a floating IP namespace, while instances without floating IPs still use centralised SNAT on a network node. It improves scale and latency, but it adds complexity, more namespaces, more agents and harder debugging, and it requires external connectivity on compute nodes. With OVN this distributed behaviour is built in, so DVR is mainly a topic for ML2 OVS clouds. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- Which traffic still goes through the network node?
- How does OVN change this?

DVR runs routers on compute nodes so east-west and floating IP traffic avoid the network node bottleneck. Centralised SNAT remains for instances without floating IPs. OVN does this natively.

```
openstack router set --distributed r1
```

```
ip netns list | grep qrouter
```

```
ip netns exec qrouter-<id> ip a
```

Interview tip: Say what remains centralised, which shows depth.

📚 Official docs

- OpenStack Neutron
- Neutron: OVN driver
- Neutron: DVR with OVS HA

certs.eknathalabs.com

Q37 How do floating IPs and SNAT work for instances?

Neutron

🎙 Spoken answer · ~59 s

A private instance sends traffic to its subnet gateway, which is a Neutron router. For outbound traffic without a floating IP, the router performs source NAT, replacing the instance address with the router's external gateway address, so many instances share one public IP. A floating IP is a one-to-one NAT mapping: Neutron associates a public address from an external network pool with the instance port, and the router performs DNAT for inbound traffic and SNAT for outbound traffic using that address. The instance itself never sees the public address, only its private one. Security groups still apply to the private port, so you must allow inbound traffic there. For troubleshooting I check the router namespace NAT rules, the external gateway, the instance security group, and whether the upstream network knows how to reach the floating range, often through ARP or BGP.

Likely follow-ups

- Why do floating IPs fail even when the association exists?
- How is the floating pool routed upstream?

Routers SNAT outbound traffic and floating IPs add one-to-one DNAT and SNAT. Instances keep private addresses and security groups apply to the private port.

```
openstack floating ip create public
```

```
openstack server add floating ip web1 203.0.113.10
```

```
ip netns exec qrouter-<id> iptables -t nat -S | grep 203.0.113.10
```

Interview tip: Remember to say the upstream network must route the floating range.

📚 Official docs

- RFC 3022: Traditional NAT
- Neutron: Self-service networks
- Nova documentation

certs.eknathalabs.com

Q38 How do security groups work and how are they implemented?

Neutron

🎙 Spoken answer · ~60 s

A security group is a set of stateful allow rules attached to a port. By default all inbound traffic is denied and all outbound is allowed, and you add rules by direction, protocol, port range, and either a CIDR or another security group as the source, which is very useful for tier-to-tier rules such as allowing the app group to reach the database group. Because they are stateful, return traffic is permitted automatically through connection tracking. In ML2 OVS, rules become iptables or OVS flows with conntrack on the compute host, in the native OVS firewall driver, and in OVN they are ACLs translated into logical flows. Port security also adds anti-spoofing, preventing a guest from sending packets with another MAC or IP address. For debugging, I check the rules, the attached groups on the port, and the conntrack entries on the host.

Likely follow-ups

- Why reference a security group as a source?
- How do you disable port security for an NFV instance?

Stateful allow-only rules attached to ports, with default deny inbound. Implemented as conntrack-based OVS flows or OVN ACLs; port security blocks spoofing.

```
openstack security group rule create --protocol tcp --dst-port 443 --remote-group app-sg db-sg
```

```
openstack port show <port> -c security_group_ids
```

```
conntrack -L | grep <ip>
```

Interview tip: Mention group-to-group rules, which scale better than CIDR lists.

📚 Official docs

- Neutron documentation

certs.eknathalabs.com

Q39 What is Octavia and how does it provide load balancing as a service?

Neutron

🎙 Spoken answer · ~62 s

Octavia is the load-balancing service for OpenStack. When a user creates a load balancer, a listener, a pool and members, the Octavia controller builds an amphora, which is a small virtual machine running HAProxy and keepalived, in a management network, and configures it through a REST agent. In active-standby topology two amphorae share a virtual IP using VRRP, so a failure of one is handled automatically. Octavia supports L4 and L7 policies, TLS termination with certificates stored in Barbican, health monitors, and since newer releases, a provider framework with OVN as a lightweight L4 provider that avoids amphora VMs. For Kubernetes on OpenStack, the cloud provider uses Octavia to create service type LoadBalancer. Operationally I watch amphora health, image updates, the management network, and quota for the load balancer project. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- What is an amphora?
- When would you use the OVN provider instead?

Octavia creates HAProxy-based amphora VMs, optionally in active-standby with VRRP, and offers L4/L7, TLS termination and health monitors. The OVN provider is a lightweight L4 option.

```
openstack loadbalancer create --vip-subnet-id web-subnet --name lb1
```

```
openstack loadbalancer listener create --protocol HTTPS --protocol-port 443 lb1
```

```
openstack loadbalancer amphora list
```

Interview tip: Link Octavia to Kubernetes services of type LoadBalancer.

📚 Official docs

- Cloudflare Learning: Load balancing
- Octavia documentation

certs.eknathalabs.com

Q40 An instance does not get an IP address from DHCP. How do you troubleshoot? Neutron

🎙 Spoken answer · ~58 s

I work from the guest outward. First I check the instance console log to see whether the DHCP client ran and what it sent. Second, I confirm the port is ACTIVE and bound to the expected host, because a port in DOWN or binding failed means the data plane is not wired. Third, I verify the DHCP agent or OVN DHCP is alive for that network, that the subnet has DHCP enabled and free addresses, and that the port's MAC and IP are registered. Fourth, I capture traffic on the tap interface, the integration bridge and the dnsmasq namespace, looking for the DISCOVER and OFFER. Common causes are security group rules or port security blocking, an overlay MTU problem, a down agent, an exhausted subnet, or a mismatched VLAN on a provider network. I fix the specific layer and retest.

Likely follow-ups

- How do you capture on a tap device?
- What if only some instances fail?

Check console log, port status and binding, DHCP agent and subnet state, then capture DISCOVER/OFFER on tap, bridge and namespace. Usual culprits: agent down, binding failure, MTU, VLAN, exhausted pool.

```
openstack port show <port> -c status -c binding_host_id
```

```
ip netns exec qdhcp-<net> tcpdump -ni any port 67 or port 68
```

```
openstack subnet show <subnet> -c allocation_pools
```

Interview tip: Narrate layer by layer: guest, port, agent, wire.

📚 Official docs

- Nova documentation
- Neutron documentation

certs.eknathalabs.com

Q41 What does a Neutron port binding failure mean and how do you fix it?

Neutron

🎙 Spoken answer · ~58 s

When an instance boots, Nova asks Neutron to bind the port to the chosen host. Neutron's ML2 asks each mechanism driver whether it can bind that port on that host for a network of that type. If none can, the port ends in binding_failed, and the instance goes to ERROR. Typical causes are an agent that is down or not reporting on that host, a missing or misnamed physical network mapping such as bridge_mappings, a network type that the host does not support, or a mechanism driver misconfiguration, for example an SR-IOV request on a host without VFs. I read the port's binding profile and vif_type, check agent status on the host, compare the physnet name with the host configuration, and look at the neutron-server log for the reason. After fixing the cause, I rebuild or recreate the instance.

Likely follow-ups

- Where is physical network mapping configured?
- How can the host agent list help?

Binding fails when no mechanism driver can bind the port on that host: agent down, wrong physnet mapping or unsupported network type. Inspect vif_type, agent state and server logs.

```
openstack port show <port> -c binding_vif_type -c binding_host_id
```

```
openstack network agent list --host <host>
```

```
grep -i 'binding' /var/log/neutron/neutron-server.log | tail
```

Interview tip: Say vif_type=binding_failed and then check agent, physnet, type.

📚 Official docs

- OpenStack Neutron
- OpenStack documentation

certs.eknathalabs.com

Q42 How do you debug network paths using namespaces, tcpdump and flows?

Neutron

🎙 Spoken answer · ~60 s

I follow the packet through each hop. On the compute host the path is guest, tap or vhost-user device, integration bridge br-int, then the tunnel bridge or provider bridge, and out to the physical NIC. I start with ip link and bridge checks, then use tcpdump on the tap and on the physical interface to see where packets stop. For routers and DHCP in ML2 OVS, I enter the network namespace with ip netns exec and run ip, iptables and tcpdump inside it. For flow-level problems, I use ovs-ofctl dump-flows and ovs-appctl ofproto/trace to simulate a packet and show which rule matches. With OVN, ovn-trace does the same for logical flows. Comparing where a packet is seen last tells me which component is dropping it. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- What does ofproto/trace give you?
- How do you capture on a vhost-user interface?

Trace hop by hop: tap, br-int, tunnel or provider bridge, NIC. Use ip netns exec for routers and DHCP, and ovs-appctl ofproto/trace or ovn-trace to find the dropping rule.

```
ovs-appctl ofproto/trace br-int in_port=tap1234,dl_src=fa:16:3e:aa:bb:cc
```

```
ovn-trace tenant-net 'inport=="port1" && eth.src==fa:16:3e:aa:bb:cc'
```

```
ip netns exec qrouter-<id> tcpdump -ni any icmp
```

Interview tip: The last hop where you still see the packet is where it gets dropped.

📚 Official docs

- pcap-filter(7)
- tcpdump manual
- Neutron documentation

certs.eknathalabs.com

Q43 What is SR-IOV and how do you use it with OpenStack?

Neutron

🎙 Spoken answer · ~63 s

SR-IOV lets a physical NIC present multiple virtual functions, each a lightweight PCIe device that can be assigned directly to a guest, bypassing the host's virtual switch. This gives near line rate throughput and low latency with little CPU use, which suits NFV, telco and high-throughput workloads. In OpenStack you enable VFs on the NIC and IOMMU in the BIOS and kernel, whitelist the physical network in the Nova PCI configuration, enable the sriovnicswitch mechanism driver in Neutron with the SR-IOV agent, and create ports with vnic_type direct. The trade-offs are significant: no software security groups on the data path in many setups, limited live migration support, hardware-specific drivers in the guest, and fewer overlay features. I use it where performance justifies those constraints, and keep a normal OVS path for everything else. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- Why is live migration hard with SR-IOV?
- What replaces security groups here?

SR-IOV assigns NIC virtual functions straight to guests for near line-rate networking. Use vnic_type direct, whitelist in Nova, and accept limits on migration and software filtering.

```
echo 8 > /sys/class/net/ens1f0/device/sriov_numvfs
```

```
openstack port create --vnic-type direct --network sriov-net sriov-port
```

```
lspci | grep -i 'virtual function'
```

Interview tip: State the trade-off: speed versus features such as migration and filtering.

📚 Official docs

- Neutron: SR-IOV
- OpenStack documentation

certs.eknathalabs.com

Q44 How does Neutron QoS work and what can it limit?

Neutron

🎙 Spoken answer · ~63 s

The QoS extension lets you define policies with rules and attach them to ports or networks. Supported rule types include bandwidth limit, which caps egress or ingress rate, minimum bandwidth, which guarantees capacity and is enforced through Placement so the scheduler considers it, DSCP marking to set packet priority, and packet rate limits. The back end enforces it: OVS uses ingress policing and queues, OVN uses meters and QoS tables, and SR-IOV applies limits on the virtual function. I use QoS to stop one tenant saturating an uplink, to provide tiered network service levels, and to protect latency-sensitive traffic with DSCP. The limitations are that enforcement depends on the driver, that ingress and egress behave differently, and that without a minimum bandwidth guarantee QoS only restricts but does not promise capacity. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- How does minimum bandwidth interact with Placement?
- Which direction is easier to limit?

QoS policies attach bandwidth, minimum bandwidth, DSCP and packet-rate rules to ports or networks. Enforcement depends on the driver, and minimum guarantees need Placement support.

```
openstack network qos policy create gold
```

```
openstack network qos rule create --type bandwidth-limit --max-kbps 100000 --egress gold
```

```
openstack port set --qos-policy gold <port>
```

Interview tip: Distinguish limits from guarantees.

📚 Official docs

- Kubernetes: Pod Quality of Service classes
- OpenStack Neutron
- Neutron: QoS

certs.eknathalabs.com

Q45 What are trunk ports and when are they needed?

Neutron

🎙 Spoken answer · ~61 s

A trunk port carries traffic for several VLANs over a single virtual NIC. The parent port belongs to one network, and subports are attached with VLAN IDs, so the guest sees tagged frames and can create VLAN subinterfaces for each. This is needed for workloads that expect to be multi-homed through VLANs, such as NFV appliances, virtual routers and firewalls, and for Kubernetes or container platforms running inside a VM where pods get their own Neutron networks, as with Kuryr. It saves you from hot-plugging many separate ports, which hit hypervisor PCI slot limits. Operationally, you create the parent port, add subports with segmentation type vlan and ID, and in the guest configure matching VLAN interfaces. I verify tags at the bridge level if packets are missing. If the interviewer pushes further, I would explain how I would test this in a staging environment before touching production.

Likely follow-ups

- How does the guest need to be configured?
- Why not just add many NICs?

A trunk carries multiple tagged VLANs on one vNIC via a parent port and subports. Used by NFV appliances and nested container networking to avoid many NICs.

```
openstack network trunk create --parent-port p0 trunk0
```

```
openstack network trunk set --subport port=p1,segmentation-type=vlan,segmentation-id=101 trunk0
```

```
ip link add link eth0 name eth0.101 type vlan id 101
```

Interview tip: Mention Kuryr or NFV as the real-world use.

📚 Official docs

- Neutron: Trunking

certs.eknathalabs.com

Q46 How do you handle IPv6 and dual stack in OpenStack?

Neutron

🎙 **Spoken answer · ~63 s**

Neutron supports IPv6 subnets with either SLAAC, DHCPv6 stateless or stateful modes, controlled by ipv6_ra_mode and ipv6_address_mode. Router advertisements are generated by the router or the OVN logical router so guests configure addresses automatically. For dual stack, a network has both an IPv4 and an IPv6 subnet and ports receive both. There is no NAT for IPv6 by default, so instances get globally routable addresses, which means security groups become the primary protection, and the upstream network must route the prefix, either through static routes or BGP dynamic routing and prefix delegation. Common problems are forgetting IPv6 rules in security groups, an upstream that does not route the tenant prefix, and images that do not run an IPv6 client. I test with ping6 and a real application, not only router advertisements. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- Why do security groups matter more for IPv6?
- What does prefix delegation provide?

IPv6 uses SLAAC or DHCPv6 with router advertisements. Dual stack gives both address families; there is no NAT, so security groups and upstream routing of the prefix are essential.

```
openstack subnet create --ip-version 6 --ipv6-ra-mode slaac --ipv6-address-mode slaac --
network n1 --subnet-range 2001:db8:1::/64 v6
```

```
openstack security group rule create --ethertype IPv6 --protocol icmpv6 default
```

```
ping6 2001:db8:1::1
```

Interview tip: Point out that no NAT shifts protection to security groups.

📚 **Official docs**

- Neutron: IPv6
- OpenStack documentation

certs.eknathalabs.com

Q47 How does Neutron integrate with BGP and dynamic routing?

Neutron

🎙 **Spoken answer · ~65 s**

For public address space, the upstream needs to know where tenant or floating prefixes live. Neutron dynamic routing provides a BGP speaker that advertises tenant subnets and floating IPs to physical routers, so you do not maintain static routes. It uses address scopes and subnet pools to make sure only the right prefixes are advertised. With OVN, a BGP agent for OVN or an integration with FRR can advertise routes directly from each compute host, enabling routed networks where each rack is its own L3 domain, which avoids large stretched L2 segments and gives better scalability and failure isolation. I use this in leaf-spine fabrics: addresses are announced from where they live, ECMP gives load balancing, and failure convergence is handled by the routing protocol instead of VRRP or ARP tricks. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- What are address scopes for?
- Why prefer routed networks over stretched L2?

Neutron BGP speakers or OVN with FRR advertise tenant and floating prefixes to the fabric, enabling routed leaf-spine designs without stretched L2 or static routes.

```
openstack bgp speaker create --local-as 64512 --ip-version 4 bgp1
```

```
openstack bgp peer create --peer-ip 10.0.0.1 --remote-as 64513 tor1
```

```
vtysh -c 'show ip bgp summary'
```

Interview tip: Say leaf-spine with ECMP and per-rack L3 domains.

📚 **Official docs**

- RFC 4271: BGP-4
- Cloudflare Learning: What is BGP?
- OpenStack Neutron

certs.eknathalabs.com

Q48 What are Designate and DNS integration in Neutron?

Neutron

🎙 Spoken answer · ~63 s

Designate is the DNS as a service project. Users create zones and recordsets through an API, and Designate publishes them to authoritative servers such as BIND or PowerDNS through a pool configuration. Neutron integrates with it so that when a port or floating IP is created, a DNS record is created automatically, for example web1.example.org pointing to the instance address, using the dns_name and dns_domain attributes, and removed again on deletion. This avoids manual DNS and stale records, and provides internal names for instances in a project. For internal resolution inside a tenant, Neutron DHCP can also serve DNS through dnsmasq with a configured forwarder. In production I decide who owns each zone, keep tenant zones separate from platform zones, and monitor propagation and the sink service that listens to notifications. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- What is the Designate sink?
- How do you keep tenants from creating records in platform zones?

Designate offers DNS as a service publishing to BIND or PowerDNS. Neutron integration creates and removes records for ports and floating IPs automatically.

```
openstack zone create --email admin@example.org example.org.
```

```
openstack network set --dns-domain example.org. net1
```

```
openstack recordset list example.org.
```

Interview tip: Mention zone ownership and delegation for tenants.

📚 Official docs

- OpenStack Neutron
- Neutron: DNS integration
- OpenStack documentation

certs.eknathalabs.com

Q49 How do you design the physical network for an OpenStack cloud?

Neutron

🎙 Spoken answer · ~62 s

I separate traffic types, usually into management, API, tunnel or overlay, storage, and external or provider networks, either as separate VLANs on bonded NICs or on separate physical interfaces. Storage traffic, especially Ceph, gets its own high-bandwidth and ideally non-oversubscribed path, with a separate replication network. The overlay network needs a larger MTU. Each node has redundant NICs in LACP or active-backup bonds connected to two top-of-rack switches in MLAG or a leaf-spine fabric with ECMP. The control plane sits on a stable L2 or L3 segment with a virtual IP managed by keepalived. I plan IP address ranges, VLAN ranges, firewalling between planes, and capacity with oversubscription ratios. Documenting the design as a diagram and an addressing table is as important as the hardware, because troubleshooting depends on it. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- Why separate storage and tenant traffic?
- What bonding mode would you use?

Separate management, API, overlay, storage and provider traffic; bond NICs across two ToR switches; jumbo MTU for overlay and storage; plan IPs and VLANs and document them.

```
ip -d link show bond0
```

```
cat /proc/net/bonding/bond0
```

```
ethtool -S ens1f0 | grep -i drop
```

Interview tip: Lead with traffic separation, then redundancy, then MTU.

📚 Official docs

- OpenStack Architecture Design Guide
- OpenStack documentation

certs.eknathalabs.com

Q50 What is the metadata proxy path and why do some instances fail to get SSH keys?

Neutron

🎙 **Spoken answer · ~63 s**

Instances request metadata from 169.254.169.254. In ML2 OVS the request reaches a metadata proxy running in the router or DHCP namespace, which adds the instance and network identity and forwards it over a Unix socket to the neutron-metadata-agent, then to the Nova metadata API with a shared secret signature. In OVN the ovn-metadata-agent on each host serves this through a namespace and haproxy. Failures leave cloud-init with no key or user data. Causes include a missing route or DHCP option for the link-local address, the metadata agent being down, a shared secret mismatch between Neutron and Nova, a security group blocking egress to the address, or the Nova metadata API being unreachable. I curl the endpoint from inside the guest through a console, then walk the chain, agent, secret, Nova API. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- What is the shared secret for?
- How does a config drive avoid all this?

169.254.169.254 requests go through a namespace proxy and the metadata agent to Nova metadata, signed with a shared secret. Failures: agent down, secret mismatch, routing or egress blocked.

```
curl -sv http://169.254.169.254/latest/meta-data/
```

```
systemctl status neutron-metadata-agent
```

```
grep metadata_proxy_shared_secret /etc/neutron/metadata_agent.ini /etc/nova/nova.conf
```

Interview tip: Name the shared-secret mismatch, a classic cause.

📚 **Official docs**

- OpenSSH manual pages
- Nova: Metadata service
- Nova documentation

certs.eknathalabs.com

Q51 Explain the Cinder architecture and how a volume is created and attached.

Storage

🎙 Spoken answer · ~62 s

Cinder provides persistent block storage. The cinder-api accepts requests, the scheduler chooses a back end, based on capacity, volume type and extra specs, and cinder-volume runs per back end, talking to the actual storage system, such as Ceph RBD, LVM, NetApp or Pure, through a driver. When a user creates a volume, the scheduler picks a back end, and the driver allocates it. When the volume is attached to an instance, Nova asks Cinder to export it and returns connection information, then os-brick on the compute node connects it, using iSCSI, NVMe over TCP, Fibre Channel or RBD, and libvirt presents it as a virtual disk. Detach reverses this. Because the volume is independent of the instance, data survives deletion of the server. Key operational points are multiple back ends, volume types to map service classes, and keeping the driver and os-brick versions compatible with the storage array.

Likely follow-ups

- What does os-brick do?
- What is a volume type?

API, scheduler and per-backend cinder-volume with a driver. Attach goes through Nova and os-brick on the compute host. Volumes outlive instances.

```
openstack volume create --size 50 --type ceph-ssd data1
```

```
openstack server add volume web1 data1
```

```
openstack volume service list
```

Interview tip: Mention os-brick as the glue on the compute side.

📚 Official docs

- OpenStack documentation
- Cinder documentation

certs.eknathalabs.com

Q52 Why is Ceph the most common storage back end for OpenStack?

Storage

🎙 **Spoken answer · ~67 s**

Ceph provides one unified, distributed storage cluster for several OpenStack needs: RBD block devices for Cinder volumes, Nova ephemeral disks and Glance images, RGW for S3 and Swift compatible object storage, and CephFS for shared file systems with Manila. It has no single point of failure, scales horizontally by adding OSD nodes, and data is spread across the cluster with the CRUSH algorithm and protected by replication, typically three copies, or erasure coding for cost efficiency. Because Nova, Glance and Cinder share the same RBD pools, creating a volume or instance from an image is a copy-on-write clone that is almost instant, and live migration works without copying disks. The costs are operational complexity, network requirements, and the need for good capacity planning and monitoring of placement groups, recovery traffic and OSD health. I would finish by naming the one log line or metric that proves the change worked, because verification is what separates a confident answer from a guess.

Likely follow-ups

- How does CRUSH choose where data lives?
- What are the downsides?

Ceph unifies block, object and file storage with replication and CRUSH. Shared RBD pools give instant copy-on-write clones and shared-storage live migration. Cost is operational complexity.

```
ceph -s
```

```
ceph osd pool ls detail
```

```
rbd -p volumes ls
```

Interview tip: Say shared pools make clone-from-image instant.

📚 **Official docs**

- Ceph: Block devices and OpenStack
- OpenStack documentation
- Ceph documentation

certs.eknathalabs.com

Q53 What are volume types and how do you use them for service tiers?

Storage

🎙 Spoken answer · ~57 s

A volume type is a label that maps to a set of extra specs, which tell the scheduler and driver how to place and configure a volume. For example a type called ssd can map to a Ceph pool on NVMe and a type called hdd to a capacity pool, and the extra spec volume_backend_name chooses the back end. You can also attach QoS specs such as maximum IOPS and throughput, encryption settings, replication and multi-attach capability. Types can be public or private to specific projects, which is how you offer premium storage only to some tenants. Users select a type at creation time, and operators can retype existing volumes, with migration if the back end changes. This makes volume types the primary tool for building a storage catalogue with distinct performance, cost and protection levels.

Likely follow-ups

- How do you restrict a type to one project?
- What does retype do when the back end differs?

Volume types map to back ends and policies through extra specs and QoS. Use them to offer tiers like ssd and hdd, and to restrict premium types to specific projects.

```
openstack volume type create --property volume_backend_name=ceph-nvme fast
```

```
openstack volume qos create --consumer front-end --property read_iops_sec=5000 fast-qos
```

```
openstack volume type set --private fast
```

Interview tip: Present types as your storage catalogue.

📚 Official docs

- Cinder documentation

certs.eknathalabs.com

Q54 What is boot from volume and what are its advantages?

Storage

🎙 Spoken answer · ~59 s

With boot from volume, the instance root disk is a Cinder volume rather than an ephemeral disk on the hypervisor. You create it from an image, and Nova attaches it as the boot device. The big advantages are persistence and mobility. The disk survives instance deletion if you keep the volume, it can be snapshotted and backed up with Cinder tools, and the instance can be evacuated or live-migrated without copying the disk when the storage is shared. It also allows larger root disks than local flavors, and boot is quick with Ceph clones. The downsides are dependence on the storage network and Cinder availability, since a storage outage now means the instances' root disks are unavailable, and slightly more complex quota management for volumes. For production pets and databases I favour boot from volume; for stateless cattle, ephemeral is fine.

Likely follow-ups

- What is delete_on_termination?
- When is ephemeral storage better?

The root disk is a Cinder volume, so it persists, can be snapshotted and makes evacuation easy. It depends on the storage backend. Ephemeral still fits stateless instances.

```
openstack server create --volume ubuntu-vol --flavor m1.small --network n1 web1
```

```
openstack volume create --image ubuntu-22.04 --size 30 ubuntu-vol
```

```
openstack server create --block-device-mapping vda=<id>:::true ...
```

Interview tip: Contrast pets and cattle to justify the choice.

📚 Official docs

- Cinder documentation

certs.eknathalabs.com

Q55 How do Cinder snapshots and backups differ?

Storage

🎙 **Spoken answer · ~57 s**

A snapshot is a point-in-time copy kept on the same storage back end, usually implemented as an efficient copy-on-write reference, so it is fast and space cheap but it shares the fate of the storage system. If the Ceph cluster is lost, the snapshots are lost too. A backup is a copy made by the Cinder backup service to a separate target, such as Swift, S3, NFS or another Ceph cluster, optionally incremental, so it protects against back-end failure, accidental deletion and corruption. I use snapshots for quick rollbacks before changes and cloning, and backups for real recovery and retention policies. For consistent backups of databases I quiesce the application or use a crash-consistent approach with a clear understanding of what is guaranteed, and I regularly test restore, because an untested backup is only a hope.

Likely follow-ups

- What are incremental backups?
- How do you get application-consistent snapshots?

Snapshots are fast, same-backend point-in-time copies. Backups go to a separate target and survive backend loss. Use both, and test restores.

```
openstack volume snapshot create --volume data1 snap1
```

```
openstack volume backup create --incremental --name bk2 data1
```

```
openstack volume backup restore bk2 data1-restored
```

Interview tip: One-liner: snapshots share the fate of the backend; backups do not.

📚 **Official docs**

- OpenStack documentation
- Cinder documentation

certs.eknathalabs.com

Q56 A volume is stuck in attaching, detaching or error_deleting. How do you recover?

Storage

🎙 **Spoken answer · ~59 s**

First I find out whether the problem is in Nova, Cinder or the storage back end. I check the volume's attachment records in Cinder and the block device mappings in Nova, because most stuck states come from these two disagreeing. I read the cinder-volume and nova-compute logs by request ID, and check whether the connection really exists on the host, using iscsiadm, multipath or rbd status. If the backend operation has completed but the state was not updated, I use the reset-state admin command for the volume or the attachment to move it to a consistent state, and then detach or delete normally. If a back-end volume is truly gone or locked, I fix it on the storage system. I avoid editing the database by hand, and I add an alert on volumes stuck in transitional states longer than a threshold.

Likely follow-ups

- What does reset-state do?
- How do you find orphan attachments?

Compare Cinder attachments with Nova block device mappings, read logs by request ID, check the host connection, then use reset-state to align them. Alert on long transitional states.

```
openstack volume show data1 -c status -c attachments
```

```
cinder reset-state --state available data1
```

```
cinder attachment-list --volume-id <id>
```

Interview tip: Say Nova and Cinder disagreeing is the root of most stuck states.

📚 **Official docs**

- Cinder documentation

certs.eknathalabs.com

Q57 What is multi-attach and when is it safe to use?

Storage

🎙 Spoken answer · ~65 s

Multi-attach lets one Cinder volume be attached to more than one instance at the same time. It must be enabled through a volume type with the multiattach property, and the back end must support it. It is not magic shared storage: a normal file system such as ext4 or XFS will be corrupted if two hosts mount it read-write. It is safe only with a cluster-aware file system, like GFS2 or OCFS2, with a cluster manager, or in read-only use cases, for example distributing a shared dataset read-only to many nodes. It is also used for some shared-disk database or clustering software that handles fencing, such as active-passive failover with SCSI reservations. If you need a shared file system for general use, Manila with CephFS or NFS is the right answer, not multi-attach. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- Why does ext4 corrupt under multi-attach?
- When is Manila the better choice?

One volume attached to several instances, via a multiattach volume type. Only safe with cluster-aware filesystems or read-only use; for general shared files use Manila.

```
openstack volume type create --property multiattach='<is> True' shared
```

```
openstack volume create --type shared --size 20 shared1
```

```
openstack server add volume node2 shared1
```

Interview tip: Say clearly that multi-attach does not equal shared filesystem.

📚 Official docs

- Cinder documentation

certs.eknathalabs.com

Q58 How does Swift object storage work: rings, replicas and consistency?

Storage

🎙 Spoken answer · ~57 s

Swift stores objects, which are files with metadata, in containers under accounts, accessed over HTTP. Proxy servers accept requests and use a ring, a consistent-hashing structure, to determine which storage nodes hold each partition. Objects are replicated, usually three times, across zones, and the account, container and object servers on storage nodes store the data. Swift favours availability and eventual consistency: a write succeeds when a quorum of replicas confirm it, and background replicators and auditors repair missing or corrupted data. Adding capacity means rebalancing the ring, which moves a controlled number of partitions to avoid overload. Swift suits large unstructured data such as backups, media and archives, and it scales out linearly, but it is not a file system and not for low-latency block use. Many clouds now use Ceph RGW instead for S3 compatibility.

Likely follow-ups

- What is the ring?
- Why is rebalancing done gradually?

Proxies use a consistent-hashing ring to place replicas across zones; replicators and auditors provide eventual consistency and repair. Scales linearly for object data.

```
swift stat
```

```
swift-ring-builder object.builder
```

```
openstack container create backups
```

Interview tip: Explain the ring as consistent hashing with zones.

📚 Official docs

- Swift documentation

certs.eknathalabs.com

Q59 What is Manila and how does it differ from Cinder?

Storage

🎙 Spoken answer · ~67 s

Manila is the shared file system service. Where Cinder gives a block device that one instance, normally, formats and owns, Manila gives a network file system share, NFS, CIFS or CephFS, that many instances can mount at the same time. A user creates a share of a share type, grants access by IP, user or cephx key, and mounts it from inside the guest. Manila drivers manage back ends, such as CephFS, NetApp, or a generic driver that builds a share from a Cinder volume in a service instance. Use Manila for shared content, home directories, build caches and Kubernetes ReadWriteMany volumes. Because share traffic travels over a network the instance can reach, the network design needs a path from tenant networks to the storage network, and security is handled through access rules. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- How do you give a tenant network access to CephFS?
- When is Cinder still the better choice?

Manila provides shared NFS, CIFS or CephFS file systems with access rules, for many-instance sharing. Cinder gives a private block device per instance.

```
openstack share create --share-type cephfs NFS 100 --name data
```

```
openstack share access create data ip 10.0.0.0/24
```

```
openstack share export location list data
```

Interview tip: Use ReadWriteMany for Kubernetes as the practical example.

📚 Official docs

- OpenStack documentation
- Cinder documentation
- Manila documentation

certs.eknathalabs.com

Q60 How do you size and monitor a Ceph cluster for OpenStack?

Storage

🎙 Spoken answer · ~57 s

I size by usable capacity after replication, which for three replicas is one third of raw, and I keep a healthy margin, typically staying below about 70 to 75 percent so recovery after a failure has room. For performance I count OSDs and device types, NVMe for latency sensitive pools and HDD for capacity, since IOPS scale with the number of OSDs. The network matters just as much: a separate cluster network for replication, at least 25 GbE for dense nodes, and no oversubscription. I set failure domains, host or rack, in CRUSH so replicas never share one. For monitoring I watch cluster health, near-full warnings, PG states, slow ops, OSD latency, recovery rate and client IOPS, using the Ceph dashboard and Prometheus exporters. I also plan the impact of recovery on client I/O with throttles.

Likely follow-ups

- Why keep free capacity headroom?
- What are placement groups?

Size on usable capacity after replication with 25 to 30 percent headroom, separate public and cluster networks, set failure domains in CRUSH, and monitor health, slow ops and recovery.

```
ceph df
```

```
ceph health detail
```

```
ceph osd perf
```

Interview tip: Say headroom is needed so a node failure can re-replicate.

📚 Official docs

- OpenStack Architecture Design Guide
- Ceph: Block devices and OpenStack
- OpenStack documentation

certs.eknathalabs.com

Q61 What are ephemeral disks, and how do Nova image backends affect them?

Storage

🎙 Spoken answer · ~57 s

An ephemeral disk is storage that belongs to the instance and disappears when the instance is deleted. Nova's image backend setting decides where the root and ephemeral disks live: raw or qcow2 files on local disk in the default setup, LVM volumes, or RBD when Ceph is used. With local qcow2, the base image is cached on the host and each instance has a thin overlay, so boots are quick after the first, but the disk is tied to the host, so live migration needs block migration and host loss means data loss. With RBD, the disk is on shared storage, which enables fast live migration and evacuation. Flavors define root and ephemeral sizes. I use local ephemeral for scratch and stateless workloads where speed matters, and avoid it for anything whose data I cannot lose.

Likely follow-ups

- What happens to local disk data on host failure?
- How do you choose the image backend?

Ephemeral disks live and die with the instance; the image backend (qcow2, LVM or RBD) decides where. Local is fast but host-bound; RBD enables shared-storage migration.

```
crudini --get /etc/nova/nova.conf libvirt images_type
```

```
openstack flavor show m1.small -c disk -c 'OS-FLV-EXT-DATA:ephemeral'
```

```
virsh domblklist <instance>
```

Interview tip: Tie the backend choice to your evacuation and migration strategy.

📚 Official docs

- Terraform: Sensitive data in state
- OpenStack Nova
- OpenStack documentation

certs.eknathalabs.com

Q62 How do you encrypt volumes and protect data at rest?

Storage

🎙 Spoken answer · ~59 s

Cinder supports volume encryption through an encrypted volume type. When a volume of that type is created, a key is generated and stored in a key manager, Barbican, and the compute host encrypts the data using dm-crypt with LUKS before it is written to the back end. The storage system therefore only holds ciphertext, and keys are not stored with the data, so a stolen disk or an administrator on the array cannot read it. Keys can be wrapped by an HSM-backed key manager for stronger protection. The costs are some CPU overhead, the need to run a highly available Barbican, and the fact that loss of the key means loss of data, so key backup is crucial. At the Ceph layer, OSD encryption protects physical disks, and TLS protects data in transit. I combine them depending on the threat model.

Likely follow-ups

- Where is the encryption performed?
- What happens if Barbican is down?

Encrypted volume types use LUKS on the compute host with keys in Barbican, so the storage backend only sees ciphertext. Protect and back up the key manager.

```
openstack volume type create --encryption-provider luks --encryption-cipher aes-xts-plain64
--encryption-key-size 256 --encryption-control-location front-end LUKS
```

```
openstack volume create --type LUKS --size 10 secure1
```

```
openstack secret list
```

Interview tip: Remember: lose the key, lose the data.

📚 Official docs

- Kubernetes: Encrypting Secret data at rest
- Cinder documentation
- Barbican documentation

certs.eknathalabs.com

Q63 How do you migrate or retype volumes between back ends without downtime? Storage

🎙 Spoken answer · ~57 s

Cinder can move a volume between back ends with retype and migration. Retype changes the volume type, and if the target type lives on a different back end, you choose a migration policy on-demand, and Cinder copies the data. For an in-use volume, Nova coordinates the swap through a block-copy operation so the instance keeps running, using a mirror process in libvirt, which usually works but is I/O heavy. For available volumes it is a simple copy by the drivers, or an optimised driver-assisted migration if both back ends are the same vendor. I plan it in the maintenance window, rate-limit if possible, ensure enough free capacity on the target, check driver compatibility and retype support, and validate data integrity and attachments afterward. For big moves, I stage in batches and monitor performance impact on tenants.

Likely follow-ups

- Why is migrating an in-use volume riskier?
- How do you limit the I/O impact?

Retype with migration-policy on-demand copies data to the new backend; in-use volumes use a live block copy coordinated with Nova. Plan capacity, batch and verify.

```
openstack volume set --type ssd --retype-policy on-demand data1
```

```
cinder migrate data1 host@lvm2#pool
```

```
openstack volume show data1 -c migration_status
```

Interview tip: Say in-use migrations are heavy and should be batched.

📚 Official docs

- Cinder documentation

certs.eknathalabs.com

Q64 What does Cinder replication and consistency-group support offer for DR?

Storage

🎙 Spoken answer · ~57 s

Some Cinder back ends support volume replication, where the array asynchronously or synchronously mirrors a volume to a secondary array, and the cinder admin can fail over a back end to the replica if the primary site is lost. You enable it with a replication-enabled volume type, and failover promotes the secondary volumes so instances can attach them. Consistency groups, now generic volume groups, let you snapshot or replicate several related volumes at a single point in time, which matters for databases whose data and logs are on separate volumes. The limitations are driver dependence, the need to rebuild or re-attach instances at the DR site, and the operational drill to fail back. I combine array replication with application-level replication and backups, and I rehearse DR regularly, because untested failover plans tend to fail in real incidents.

Likely follow-ups

- Why do you need group snapshots for databases?
- What are the limits of Cinder failover?

Replication-enabled volume types mirror to a secondary array, and failover promotes the replica. Groups give crash-consistent multi-volume snapshots. Driver-dependent; rehearse failover.

```
openstack volume type set --property replication_enabled='<is> True' replicated
```

```
cinder failover-host host@backend
```

```
openstack volume group snapshot create group1
```

Interview tip: Stress rehearsal: failover plans must be tested.

📚 Official docs

- OpenStack documentation
- Cinder documentation

certs.eknathalabs.com

Q65 How do you provide storage QoS so noisy neighbours do not hurt others?

Storage

🎙 **Spoken answer · ~65 s**

There are two layers. In Cinder, QoS specs attached to volume types set limits such as read and write IOPS or bandwidth, applied by the hypervisor through libvirt for front-end consumers, or by the storage array itself for back-end consumers. The values can be fixed or scale with the volume size, for example IOPS per gigabyte, which makes pricing and fairness predictable. In Ceph, you can use RBD QoS settings per pool or per image, and mClock scheduling for recovery versus client traffic. For ephemeral disks, Nova flavor extra specs can set disk IO limits. I design tiers with explicit guarantees and ceilings, monitor the top consumers using per-volume latency metrics, and use alerts when a tenant approaches limits, so the noisy neighbour is visible before others complain. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- What is the difference between front-end and back-end QoS?
- How do you find the noisy neighbour?

Cinder QoS specs on volume types cap IOPS and throughput at hypervisor or array level; Ceph RBD QoS and flavor limits cover other paths. Monitor top consumers by latency.

```
openstack volume qos create --consumer front-end --property total_iops_sec=3000 tier-gold
```

```
openstack volume qos associate tier-gold gold
```

```
ceph config set global rbd_qos_iops_limit 2000
```

Interview tip: Mention per-GB scaling to keep tiers fair.

📚 **Official docs**

- Kubernetes: Multi-tenancy
- Kubernetes: Pod Quality of Service classes
- Neutron: QoS

certs.eknathalabs.com

Q66 How does Keystone authentication work and what are Fernet tokens?

Identity

🎙 Spoken answer · ~60 s

A user or service authenticates to Keystone with credentials, such as a password, application credential or federated identity, and receives a token, which is a bearer proof of identity scoped to a project, domain or system. Clients pass this token in the X-Auth-Token header to other services, which validate it with Keystone, using keystonemiddleware, and cache the result. Fernet tokens are the default type: they are encrypted, signed, and contain the user and scope, so Keystone does not need to store them in the database, which removes the old database bloat and replication cost. The signing and encryption keys are kept in a key repository, and you rotate them regularly: a new primary key is created, the old primary becomes a secondary, and the key set must be distributed to every Keystone node, otherwise tokens issued by one node cannot be validated by another.

Likely follow-ups

- Why must Fernet keys be synchronised?
- What is the token lifetime trade-off?

Credentials produce a scoped bearer token. Fernet tokens are encrypted and not stored in the DB; rotate keys regularly and distribute them to every Keystone node.

```
openstack token issue
```

```
keystone-manage fernet_rotate --keystone-user keystone --keystone-group keystone
```

```
ls -l /etc/keystone/fernet-keys/
```

Interview tip: Warn about key distribution: unsynchronised keys cause random 401s.

📚 Official docs

- Keystone: Fernet tokens
- OpenStack documentation
- Keystone documentation

certs.eknathalabs.com

Q67 Explain domains, projects, users, groups and roles in Keystone.

Identity

🎙 **Spoken answer · ~65 s**

A domain is the top-level container for identity isolation. It owns users, groups and projects, and lets different organisations or departments share one cloud without seeing each other's identities. A project is the unit that owns resources and quotas, formerly called a tenant. Users and groups are identities, and roles are named permissions, such as member, reader and admin. Access is granted by creating a role assignment: a role for a user or group on a project, domain or the system. Roles are interpreted by each service's policy file, so the word admin only means something because policies check for it. The trend in recent releases is secure RBAC, with reader, member and admin roles and system, domain and project scopes, to implement least privilege, instead of a single global admin. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- What is the difference between project and domain scope?
- What is a system-scoped token?

Domains isolate identity namespaces, projects own resources, users and groups hold roles via role assignments on a scope. Policies in each service decide what roles allow.

```
openstack domain create finance
```

```
openstack project create --domain finance payments
```

```
openstack role add --user alice --project payments member
```

Interview tip: Say roles are only names; service policies give them meaning.

📚 **Official docs**

- OpenStack documentation
- Keystone documentation

certs.eknathalabs.com

Q68 What is secure RBAC and why is it replacing the global admin?

Identity

🎙 Spoken answer · ~56 s

Historically, the admin role was a global superuser: if you had admin on any project, many services treated you as an administrator of the entire cloud. Secure RBAC introduces three personas and three scopes. The roles are reader, member and manager or admin, and the scopes are project, domain and system. Project-scoped users operate inside their project, domain-scoped users manage a domain, and system-scoped users administer the infrastructure. Each service's default policy was rewritten in that model, so a reader can view but not change, a member can manage resources in their project, and only system admins can touch cloud-wide resources like hypervisors. This fixes privilege creep, supports delegation, and enables read-only operations and audit accounts. Migrating requires testing automation and custom policies, because old tokens and scripts that assumed global admin will break.

Likely follow-ups

- How do you migrate existing automation?
- What does system scope mean?

Secure RBAC defines reader, member and admin roles across project, domain and system scopes, ending the all-powerful global admin and enabling least privilege.

```
openstack role assignment list --system --user ops1
```

```
openstack --os-system-scope all hypervisor list
```

```
openstack role add --user auditor --project payments reader
```

Interview tip: Mention that automation assuming global admin is what breaks.

📚 Official docs

- Kubernetes: RBAC
- Keystone: Service API protection (secure RBAC)

certs.eknathalabs.com

Q69 How does federated identity work with SAML or OIDC in Keystone?

Identity

🎙 Spoken answer · ~58 s

Keystone can act as a service provider that trusts an external identity provider, such as a corporate SAML IdP, Keycloak or an OIDC provider. The user authenticates at the IdP, and Keystone receives assertions or claims, then maps them to local users, groups and projects using mapping rules. A mapping, written as JSON, says for example that members of a given claim value become a local group, and that group has roles on a project. Keystone issues a normal scoped token after the mapping. This gives single sign-on, central lifecycle management, MFA enforced by the IdP, and no duplicated passwords in the cloud. In practice, the web flow works in Horizon, and the CLI uses an OIDC plugin for device or password flows. Debugging mostly means testing attribute names and mapping rules, and checking clock skew and certificates.

Likely follow-ups

- What is a mapping rule?
- How does the CLI authenticate with OIDC?

Keystone trusts an external IdP via SAML or OIDC and maps claims to local groups and roles with JSON mapping rules, giving SSO and centralised MFA.

```
openstack identity provider create --remote-id https://idp.example.org corp
```

```
openstack mapping create --rules rules.json corp-map
```

```
openstack federation protocol create --identity-provider corp --mapping corp-map openid
```

Interview tip: Say the mapping rules are where most debugging happens.

📚 Official docs

- GitHub Actions: OpenID Connect
- OAuth 2.0 / OIDC overview (OpenID)
- Keystone: Federation

certs.eknathalabs.com

Q70 How would you integrate Keystone with LDAP or Active Directory?

Identity

🎙 Spoken answer · ~56 s

Keystone supports domain-specific back ends, where one domain is backed by LDAP and others by SQL, so you can read users and groups from the directory without copying them. You configure the LDAP URL, bind account, search base and attribute mappings in a domain configuration file or through the domain config API. Keystone treats the LDAP back end as read-only for identity, so users cannot be created in it through OpenStack, and it uses the LDAP server for password checking. Role assignments are still stored in the Keystone SQL database. Good practice is to use LDAPS, a limited-privilege bind user, user and group filters to limit the scope, paging for large directories, and caching to reduce load. Many teams now prefer federation with an IdP over direct LDAP because it supports MFA and modern flows.

Likely follow-ups

- What stays in SQL when using LDAP?
- Why prefer federation today?

Domain-specific LDAP back ends read users and groups from the directory read-only, while role assignments stay in SQL. Use LDAPS, filters and caching; federation is often preferred.

```
openstack domain create corp
```

```
cat /etc/keystone/domains/keystone.corp.conf
```

```
openstack user list --domain corp
```

Interview tip: Say role assignments remain in the Keystone database.

📚 Official docs

- OpenStack documentation
- Keystone documentation

certs.eknathalabs.com

Q71 What are application credentials and when should automation use them?

Identity

🎙 Spoken answer · ~58 s

An application credential is a secret that a user creates for a specific project, which lets a script or service authenticate without using the user's password. It has an ID, a secret, optional roles, an expiry, and can be restricted, for example to non-privileged use or to specific access rules limited to certain API paths. Because it belongs to the user but is scoped and revocable, it is safer than sharing a password, it survives password changes, and it works for federated users who have no local password. Applications and CI systems use it in clouds.yaml. Good practice is to give each application its own credential with only the roles it needs, set an expiry, rotate it, store it in a secret manager, and revoke it immediately if it leaks. Trusts serve a different purpose, delegating roles between users.

Likely follow-ups

- How do access rules restrict an application credential?
- How do trusts differ?

Application credentials let automation authenticate with a scoped, revocable, expiring secret instead of a user password, and they work for federated users. One per application, least roles.

```
openstack application credential create --role member --expiration 2026-12-31T00:00:00 ci-bot
```

```
openstack application credential list
```

```
openstack --os-auth-type v3applicationcredential server list
```

Interview tip: Say one credential per application with an expiry.

📚 Official docs

- Keystone: Application credentials

certs.eknathalabs.com

Q72 How do you troubleshoot intermittent 401 errors across OpenStack APIs?

Identity

🎙 Spoken answer · ~67 s

Intermittent authentication failures usually come from inconsistency between nodes rather than a wrong password. The first suspect is Fernet key mismatch: if the key repository differs between Keystone nodes, tokens issued by one are rejected by another, so requests succeed or fail depending on which node the load balancer picks. The second is clock skew between controllers, which makes tokens appear expired or not yet valid, so I check NTP or chrony. Third, token cache problems in keystonemiddleware through memcached, such as a failing member in the pool, can cause stale results. Fourth, expired or revoked tokens in long-running scripts. I reproduce by issuing and using tokens repeatedly, look at Keystone and service logs for the validation error, compare key files with checksums on all nodes, and verify time sync and memcached health. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- How do you compare Fernet keys across nodes?
- Which memcached issues cause 401s?

Look for Fernet key drift between Keystone nodes, clock skew, and memcached cache failures. Compare key checksums, check chrony and the middleware cache.

```
md5sum /etc/keystone/fernet-keys/* | sort
```

```
chronyc tracking
```

```
grep -i 'could not validate\|401' /var/log/keystone/*.log | tail
```

Interview tip: The 'works on some requests' symptom points to node inconsistency.

📚 Official docs

- OpenStack documentation
- Keystone documentation

certs.eknathalabs.com

Q73 How do policies work in OpenStack and how do you customise them safely?

Identity

🎙 Spoken answer · ~66 s

Each service has a policy that maps API actions to rules, such as os_compute_api:servers:create requiring the member role on the project. Defaults are defined in code, and operators override only what they need in a policy.yaml file read by oslo.policy. Rules can combine roles, scope checks, and ownership such as project_id matching the target. Since release defaults moved to secure RBAC, it is best to change as little as possible and rely on the defaults, because custom policies drift and become expensive to upgrade. To customise safely, I generate the effective policy with oslopolicy-sample-generator, change a minimal set of rules, test with a real token for each persona, put the file under version control, and review it at every upgrade for deprecated rules. I avoid granting broad admin to solve a narrow need. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- How do you see the effective policy?
- Why keep overrides minimal?

oslo.policy maps API actions to role and scope rules; defaults live in code and operators override via policy.yaml. Keep overrides minimal, test each persona and review at upgrade.

```
oslopolicy-sample-generator --namespace nova --format yaml
```

```
oslopolicy-checker --config-file /etc/nova/nova.conf --access creds.json --policy
/etc/nova/policy.yaml
```

```
git diff policy.yaml
```

Interview tip: Minimal, version-controlled overrides survive upgrades.

📚 Official docs

- OpenStack documentation
- Keystone documentation

certs.eknathalabs.com

Q74 How do service users, service tokens and the service project work?

Identity

🎙 **Spoken answer · ~58 s**

OpenStack services need identities to talk to each other, for example Nova calling Neutron or Cinder. Each service has its own Keystone user, typically in a dedicated service project, with an admin or service role. When a user request triggers a service-to-service call, services can forward the user's token. But user tokens can expire in the middle of a long operation such as a large snapshot or an upload, so service tokens were introduced: the service sends the user's token together with its own service token, and keystonemiddleware accepts an expired user token if the service token is valid. This avoids failures in long-running tasks without making token lifetime very long. I keep service credentials in a secret manager or in the deployment tool, rotate them, and use distinct users per service so access can be audited and revoked individually.

Likely follow-ups

- Why do long operations fail with plain user tokens?
- How do you rotate service passwords?

Each service has its own Keystone user in a service project. Service tokens accompany user tokens so long operations survive user-token expiry. Use distinct, rotatable credentials.

```
openstack user list --domain default | grep -E 'nova|neutron|cinder'
```

```
grep -A6 service_user /etc/nova/nova.conf
```

```
openstack role assignment list --user nova --project service
```

Interview tip: Mention long-running snapshots as the reason service tokens exist.

📚 **Official docs**

- Keystone: Fernet tokens
- Keystone documentation

certs.eknathalabs.com

Q75 How would you make Keystone highly available and performant?

Identity

🎙 Spoken answer · ~63 s

Keystone is stateless apart from its database and the Fernet key repository, so you run several Keystone instances behind HAProxy and a virtual IP, usually under Apache with mod_wsgi, or uWSGI. The database must be highly available, typically Galera, and every node needs the same Fernet and credential keys, distributed by the deployment tool on rotation. Performance depends on caching: oslo.cache with memcached for tokens validated by keystonemiddleware, for catalog and role lookups, and for revocation events, with multiple memcached servers. I size workers with the CPU count, tune database connection pools, watch authentication latency, since every API call depends on it, and keep token lifetime reasonable. For multi-region clouds, I use a shared Keystone with a replicated database or federate between regions. Rate limiting and monitoring failed logins protect against abuse. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- What caching layers matter most?
- How do you share Keystone between regions?

Run several stateless Keystone nodes behind HAProxy with Galera and identical Fernet keys, cache aggressively with memcached, and watch auth latency because everything depends on it.

```
openstack token issue --debug 2>&1 | grep -i 'elapsed'
```

```
echo stats | nc 127.0.0.1 11211 | grep -E 'get_hits|get_misses'
```

```
systemctl status apache2
```

Interview tip: Mention memcached caching, since validation volume dominates load.

📚 Official docs

- OpenStack documentation
- Keystone documentation

certs.eknathalabs.com

Q76 Compare Kolla-Ansible, OpenStack-Ansible and Charmed OpenStack as deployment tools.

Deploy

🎙 **Spoken answer · ~66 s**

Kolla-Ansible deploys every OpenStack service as a Docker container using Ansible playbooks. It is simple to reason about, easy to upgrade by pulling new images, and widely used for private clouds. OpenStack-Ansible installs services into LXC containers or directly on hosts from source with Ansible roles, which gives fine control and is common where operators want to build from upstream code. Charmed OpenStack uses Juju charms from Canonical, modelling relations between services, and works well with MAAS for bare-metal provisioning, with an opinionated, supported path. TripleO is deprecated. I choose based on team skills and support needs: Kolla for container-first simplicity, OSA for flexibility, Charms for a vendor-supported lifecycle. Whichever tool I pick, the key practices are the same: configuration in Git, repeatable inventory, tested upgrades, and never hand-editing deployed files. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- Why was TripleO retired?
- How do you manage secrets in Kolla?

Kolla-Ansible runs services as containers, OpenStack-Ansible uses LXC or source installs, Charmed uses Juju and MAAS. Pick by team skills and support needs; keep config in Git.

```
kolla-ansible -i multinode bootstrap-servers
```

```
kolla-ansible -i multinode deploy
```

```
openstack-ansible setup-everything.yml
```

Interview tip: Say configuration is in Git and playbooks are the only thing that touches hosts.

📚 **Official docs**

- Ansible documentation
- OpenStack documentation
- Kolla-Ansible documentation

certs.eknathalabs.com

Q77 Walk me through deploying a production OpenStack cloud with Kolla-Ansible.

Deploy

🎙 Spoken answer · ~66 s

I start with design: hardware roles, networks, VIPs and storage. Then I prepare hosts with a supported OS, consistent time sync, DNS, bonded NICs and VLANs, and disks for Ceph or Cinder. I create the deployment node with Ansible and Kolla-Ansible, copy the example globals.yml and passwords.yml, and generate passwords. In the inventory I assign nodes to control, network, compute, storage and monitoring groups. I set the VIP address, network interfaces, enabled services, TLS and the Docker registry or local mirror. Then I run bootstrap-servers, prechecks, pull and deploy. Afterwards I run post-deploy to create admin credentials, register a test image, flavor and network, and run a smoke test with a synthetic instance. I commit configuration to Git, set up monitoring, backup, and document the runbook, then perform failure tests before onboarding tenants. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- What do prechecks catch?
- How do you customise one service's config?

Design hardware and networks, prep hosts, write inventory and globals.yml, run bootstrap-servers, prechecks, pull, deploy and post-deploy, smoke test, then harden, monitor and back up.

```
kolla-genpwd
```

```
kolla-ansible -i multinode prechecks
```

```
kolla-ansible -i multinode post-deploy
```

Interview tip: End with failure testing before onboarding tenants.

📚 Official docs

- Google SRE Book: Effective Troubleshooting
- Ansible documentation
- OpenStack documentation

certs.eknathalabs.com

Q78 How do you size a control plane and compute fleet for a new cloud?

Deploy

🎙 Spoken answer · ~57 s

I begin with requirements: number of tenants, expected instances, API request rate, growth and availability goals. For the control plane, three nodes is the standard minimum for quorum with Galera, RabbitMQ and Pacemaker or HAProxy, each with enough CPU, RAM and fast SSD or NVMe for the database, and I scale services out onto separate nodes when load grows. Compute sizing starts from workload shapes: average vCPU to RAM ratio, overcommit targets by class, NUMA layout, local disk needs and network bandwidth per host. I reserve capacity for failure, so that losing one host or a rack still leaves room to evacuate, usually N plus one or a percentage buffer. Storage sizing is separate, based on usable capacity after replication and IOPS. I plan monitoring for real consumption and review quarterly, because forecasts are always wrong.

Likely follow-ups

- Why three controllers?
- How do you reserve failure headroom?

Start from workload and availability requirements. Three control nodes for quorum, compute sized by vCPU:RAM shapes and overcommit, reserve N+1 failure capacity, size storage separately.

```
openstack hypervisor stats show
```

```
openstack usage list --start 2026-01-01
```

```
ceph df
```

Interview tip: Mention failure headroom, not just average demand.

📚 Official docs

- Argo CD: ApplicationSet
- OpenStack Architecture Design Guide

certs.eknathalabs.com

Q79 What are the network types and VLANs you create in a standard OpenStack deployment?

Deploy

🎙 Spoken answer · ~65 s

A typical deployment separates traffic into several networks. The management network carries internal service communication, SSH and the control-plane. The API network or internal VIP serves service endpoints. The tunnel or overlay network carries VXLAN or Geneve between hypervisors. The storage network, often split into public and cluster for Ceph, carries volume traffic and replication. The external or provider network connects to the data centre and the internet for floating IPs and public services. Optional networks include an out-of-band BMC network, a provisioning network for PXE and Ironic, and a dedicated network for live migration. Each has its own IP range and VLAN, and firewall rules between them. Separating them gives security, predictable performance and easier troubleshooting, and they can share physical NICs through VLANs when the hardware is limited. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- Which networks need jumbo frames?
- Why separate the provisioning network?

Management, API, overlay, storage (public and cluster), external and optional provisioning and BMC networks, each with its own VLAN and addressing.

```
ip -br addr
```

```
bridge vlan show
```

```
cat /etc/kolla/globals.yml | grep -E 'interface|vip'
```

Interview tip: Offer to draw the networks and say which carry jumbo frames.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q80 What is Ironic and when would you provision bare metal through OpenStack?

Deploy

🎙 Spoken answer · ~65 s

Ironic is the bare-metal provisioning service. It treats physical servers as resources that Nova can schedule, using the same flavors, images and APIs as virtual machines. Ironic talks to each server's BMC through IPMI or Redfish to power it on and off, boots a deploy ramdisk over PXE or virtual media, writes the image to disk, configures networking, then boots the final OS. It also supports cleaning between tenants, which wipes disks and resets firmware, and inspection to discover hardware. I use it for workloads that need raw hardware: databases with heavy IO, HPC, GPU nodes without virtualisation overhead, and to deploy the cloud's own compute nodes, as in a hardware lifecycle pipeline. The challenges are network isolation, slower provisioning than VMs, and the need for good hardware inventory and BMC security. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- What is automated cleaning?
- How does Neutron isolate bare-metal tenants?

Ironic manages physical servers via IPMI or Redfish and PXE, exposing them through Nova flavors. Used for raw performance, HPC, GPU, and for bootstrapping compute nodes; includes cleaning and inspection.

```
openstack baremetal node list
```

```
openstack baremetal node manage node1 && openstack baremetal node provide node1
```

```
openstack server create --flavor baremetal --image ubuntu --network provision bm1
```

Interview tip: Mention automated cleaning, which proves you understand tenancy on hardware.

📚 Official docs

- OpenStack documentation
- Ironic documentation

certs.eknathalabs.com

Q81 How do you configure TLS everywhere in an OpenStack deployment?

Deploy

🎙 Spoken answer · ~56 s

I start with the public API endpoints, terminating TLS at HAProxy with certificates from the organisation's CA or Let's Encrypt, and redirect HTTP to HTTPS. Then I extend encryption to the internal VIP and the backend services, so that service-to-service traffic is encrypted too, which Kolla-Ansible supports with the internal TLS option and a CA that signs backend certificates. I also encrypt RabbitMQ connections, MariaDB replication and client connections, and Ceph messenger v2 where required. Clients and services need the CA bundle, set through cafile options and the system trust store, otherwise they fail verification. Certificate lifecycle is the hard part: I track expiry with monitoring, automate renewal and rolling reload, and document replacing the CA. TLS on all internal hops costs some CPU but removes the assumption that the internal network is trusted.

Likely follow-ups

- What breaks when a CA is not in the trust store?
- How do you rotate certificates without downtime?

TLS terminates at HAProxy for public endpoints, then extend to internal VIP, RabbitMQ and MariaDB. Distribute the CA bundle to all clients and automate renewal with expiry monitoring.

```
openssl s_client -connect cloud.example.org:443 -showcerts
```

```
openssl x509 -in /etc/kolla/certificates/haproxy.pem -noout -dates
```

```
kolla-ansible -i multinode reconfigure -t haproxy
```

Interview tip: Say that renewal automation, not initial setup, is the hard part.

📚 Official docs

- OpenStack Security Guide
- OpenStack documentation

certs.eknathalabs.com

Q82 How do you handle configuration management and secrets for your OpenStack deployment?

Deploy

🎙 Spoken answer · ~65 s

I keep the deployment as code. The inventory, globals and per-service overrides live in Git, reviewed through pull requests, and secrets such as database and service passwords live in an encrypted file or a vault, not in plain text in the repository. For Kolla-Ansible, passwords.yml is encrypted with ansible-vault or sourced from an external secret manager at deployment time. Changes are applied through the same tool, for example reconfigure for one service, so the host state always matches the repository. I avoid manual edits inside containers or on hosts because they are lost or cause drift. For auditability I tag releases, record which commit was deployed and when, and test changes in a staging cloud first. Access to the deployment host is restricted, since it holds the keys to the entire cloud. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- How do you detect configuration drift?
- Who should have access to the deploy host?

Inventory and overrides in Git, secrets in a vault, changes applied only through the deployment tool, tested in staging, with the deploy host tightly restricted.

```
ansible-vault encrypt /etc/kolla/passwords.yml
```

```
git log --oneline -- /etc/kolla/config/
```

```
kolla-ansible -i multinode reconfigure -t nova
```

Interview tip: The deploy host is a crown jewel; mention restricting it.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q83 How do you build a staging or lab cloud that mirrors production?

Deploy

🎙 Spoken answer · ~56 s

The aim is to test upgrades and changes where mistakes are cheap. A true copy is rarely affordable, so I mirror what matters: the same release and deployment tool, the same configuration repository with environment-specific variables, the same network topology in miniature, with VLANs and MTU, and the same storage back end type at a smaller scale. I automate creation so the staging cloud can be rebuilt on demand, using the same playbooks. For hardware-dependent features such as SR-IOV or GPUs, I include at least one real node. I load it with synthetic tenants and run canary and conformance tests, including Tempest, and rehearse upgrades, failover and restore procedures there. Where hardware is limited, I use nested virtualisation or virtual machines on a large host for functional testing, accepting that performance results will not transfer.

Likely follow-ups

- What does Tempest test?
- What cannot be validated in a nested lab?

Mirror release, tooling, config repo, network design and storage type at small scale; automate rebuilds; run Tempest and rehearse upgrades and failovers. Include real nodes for hardware features.

```
tempest run --smoke
```

```
kolla-ansible -i staging deploy
```

```
openstack --os-cloud staging server list
```

Interview tip: Say what you mirror (process and topology) and what you cannot (performance).

📚 Start here (official docs)

- OpenStack documentation

certs.eknathalabs.com

Q84 What is DevStack and what is it not suitable for?

Deploy

🎙 Spoken answer · ~63 s

DevStack is a script-based installer that builds a single-node or small OpenStack from upstream source, primarily for developers who want to test patches and learn the services. It is fast to start, gives access to the newest code and is the basis of the upstream CI gating environment. It is not suitable for production: it is not highly available, not hardened, not upgradeable in place, configuration is rebuilt each time, and services run in a development layout. Restarting the machine can lose state. For a production or long-lived lab I use Kolla-Ansible, OpenStack-Ansible or a vendor distribution. For a small, supported, easy to install cloud, newer options like Sunbeam or MicroStack provide a Kubernetes-based, snap-installed experience with an opinionated setup. In interviews I mention DevStack as a learning and testing tool. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- What replaces DevStack for a small cloud?
- How is DevStack used in upstream CI?

DevStack builds OpenStack from source for developers and CI. It is not HA, hardened or upgradeable, so use Kolla, OSA or a vendor distribution for anything long-lived.

```
git clone https://opendev.org/openstack/devstack
```

```
./stack.sh
```

```
./unstack.sh
```

Interview tip: Position it as a learning and patch-testing tool only.

📚 Start here (official docs)

- OpenStack documentation

certs.eknathalabs.com

Q85 How do you automate OpenStack day-two operations with Ansible and the SDK?

Deploy

🎙 Spoken answer · ~59 s

Day-two work is everything after the first deployment: onboarding projects, quotas, networks, images, patching and checks. I automate these using the openstack collection for Ansible, which has modules for projects, users, networks, security groups, flavors and servers, and openstacksdk for custom logic. A tenant onboarding playbook creates a project, assigns roles and a group, sets quotas, creates a default network and router and security group, and records everything in a ticket or inventory. Playbooks are idempotent, so rerunning is safe, and they live in Git with CI checks. For operations like draining a compute node or rotating credentials, I write tested runbooks as playbooks rather than shell snippets. This turns tribal knowledge into reviewed code, speeds up onboarding and reduces human error. If the interviewer pushes further, I would explain how I would test this in a staging environment before touching production.

Likely follow-ups

- Why must playbooks be idempotent?
- What belongs in a tenant onboarding workflow?

Use the openstack Ansible collection and openstacksdk for idempotent onboarding, quota, network and drain playbooks stored in Git with CI. Runbooks become code.

```
ansible-playbook onboard_tenant.yml -e project=payments
```

```
python drain_host.py --host compute12
```

```
ansible-galaxy collection install openstack.cloud
```

Interview tip: Say that runbooks should be executable code.

📚 Official docs

- OpenStack Operations Guide
- Ansible documentation
- OpenStack documentation

certs.eknathalabs.com

Q86 How do you provision Terraform-managed infrastructure on OpenStack?

Deploy

🎙 Spoken answer · ~61 s

The Terraform OpenStack provider exposes resources like openstack_compute_instance_v2, networking networks, subnets and routers, block storage volumes, security groups, floating IPs and load balancers. I authenticate using clouds.yaml or application credentials, never an admin password. I define a module per application stack, with variables for flavor, image and network, and keep state in a remote backend with locking. Because OpenStack names and IDs are needed, I use data sources to look up images, flavors and external networks instead of hard coding UUIDs. For consistency across clouds I wrap provider-specific parts in modules. Gotchas are replace-on-change for some arguments such as image, which recreates the instance, ports created implicitly versus explicitly, and slow operations needing proper timeouts. Pairing Terraform with cloud-init for first boot gives a clean infrastructure-as-code flow. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- Why use application credentials with Terraform?
- What causes unexpected instance replacement?

Use the OpenStack provider with application credentials, data sources for images and networks, modules per stack, remote state with locking. Watch replace-on-change arguments.

```
terraform init && terraform plan
```

```
data "openstack_images_image_v2" "ubuntu" { name = "ubuntu-22.04" }
```

```
terraform state list
```

Interview tip: Mention data sources and replacement behaviour as the two usual gotchas.

📚 Official docs

- Terraform OpenStack provider
- OpenStack documentation

certs.eknathalabs.com

Q87 What is the difference between Heat and Terraform for orchestration?

Deploy

🎙 Spoken answer · ~63 s

Heat is OpenStack's native orchestration service. You write templates in HOT, Heat Orchestration Template, YAML, describing resources, and Heat creates a stack, tracks dependencies, supports autoscaling groups and software config, and is integrated with Keystone so tenants can use it without extra tooling. It only manages OpenStack resources. Terraform is a general tool with a provider for OpenStack and for many other platforms, a large ecosystem, a plan step that previews changes, and its own state management. I choose Heat when I want tenants to deploy within the cloud with no external tooling, for autoscaling driven by Ceilometer or Aodh alarms, and for services like Magnum that use Heat internally. I choose Terraform for multi-cloud estates, mature workflows and wider team familiarity. They can coexist. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- What does Heat autoscaling use as triggers?
- How does Terraform plan help in reviews?

Heat is native, OpenStack-only, with HOT templates and autoscaling and no external state. Terraform is multi-platform with plan and state. Choose by scope and team skills.

```
openstack stack create -t web.yaml webstack
```

```
openstack stack resource list webstack
```

```
terraform plan -out=tfplan
```

Interview tip: Say Heat for in-cloud native use, Terraform for multi-cloud workflows.

📚 Official docs

- Terraform OpenStack provider
- Heat documentation

certs.eknathalabs.com

Q88 How would you bootstrap the first compute nodes using Ironic and MAAS or PXE?

Deploy

🎙 Spoken answer · ~56 s

I want repeatable, hands-off provisioning. Hardware is racked and cabled, with the BMC network configured. A provisioning service such as MAAS, or Ironic itself, discovers servers by PXE, records their hardware, and commissions them to verify disks, NICs and memory. From a defined profile it then installs the base operating system image with the right disk layout, bonding and VLANs, using cloud-init or kickstart for first-boot configuration, and registers the host in the inventory. Ansible then applies the OpenStack roles and joins the node as a compute host, and a post-checks stage verifies hypervisor, agent and network. This approach makes scaling out a matter of adding racks and re-running automation, and replacing failed nodes just means reprovisioning. I keep firmware and BIOS settings, such as virtualisation, IOMMU and boot order, in code or profile too.

Likely follow-ups

- Why record BIOS settings as code?
- What does commissioning verify?

PXE discovery and commissioning, profile-based OS install, cloud-init or kickstart for first boot, then Ansible roles join the node. Firmware and BIOS settings are codified for repeatability.

```
maas admin machines read | jq '.[].hostname'
```

```
kolla-ansible -i multinode bootstrap-servers --limit compute13
```

```
kolla-ansible -i multinode deploy --limit compute13
```

Interview tip: Make scale-out an automated process, not a runbook.

📚 Official docs

- OpenStack Nova
- Ironic documentation

certs.eknathalabs.com

Q89 What does your day-zero checklist look like before opening a cloud to tenants? Deploy

🎙 Spoken answer · ~60 s

Before onboarding anyone I check five areas. Functionality: Tempest or smoke tests pass, and a synthetic instance can boot, attach a volume, get a floating IP and be reached. Resilience: I deliberately fail a controller, a compute node, a RabbitMQ member, a Galera node and a Ceph OSD and confirm the cloud recovers as designed. Security: TLS, least-privilege roles, hardened SSH, firewalls between networks, default passwords gone, and quotas and security group defaults set. Operations: monitoring, alerting, logging, backups with a tested restore, time sync and runbooks. Capacity and documentation: network diagrams, IP plans, support contacts and an upgrade plan. Finally a tenant onboarding process, with quotas, networks, naming standards and billing or showback. Opening to tenants before failure testing is the common mistake. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- Which failure test is most revealing?
- What belongs in the tenant onboarding process?

Smoke and Tempest tests, deliberate failure tests of controllers, compute, queue, DB and Ceph, security baseline, monitoring and backups with restore, documentation and onboarding process.

```
tempest run --smoke
```

```
rabbitmqctl stop_app
```

```
openstack server list --all-projects --status ERROR
```

Interview tip: Failure testing before launch is what interviewers want to hear.

📚 Start here (official docs)

- OpenStack documentation

certs.eknathalabs.com

Q90 How would you choose between OpenStack distributions and vendor support models?

Deploy

🎙 Spoken answer · ~63 s

I compare what I need operationally against who carries the risk. Upstream-based self-managed OpenStack gives maximum flexibility and no licence cost, but my team must handle upgrades, bugs and security patches, so it needs skilled staff. Vendor distributions such as Red Hat OpenStack, Canonical Charmed OpenStack, Mirantis or SUSE offer tested combinations, long-term support, security patches, certified hardware and software ecosystems, and support contracts, at a price. Managed private cloud services from providers add operations too, which removes staff burden but also some control. The decision factors are team size and experience, compliance needs, budget, required release cadence, integration with existing storage and network vendors, and exit strategy. I also check for lock-in in the deployment tooling, and I evaluate by a proof of concept, including an upgrade exercise, before committing. I would round off by noting what I would automate here, because repeatable steps should not depend on memory.

Likely follow-ups

- What lock-in risks exist in vendor tooling?
- How do you evaluate with a PoC?

Self-managed upstream gives flexibility but needs staff. Vendor distributions bring support and tested stacks at a cost; managed services shift operations. Decide with a PoC including an upgrade.

```
openstack --version
```

```
apt-cache policy nova-compute
```

```
rpm -qa | grep openstack-nova
```

Interview tip: Frame it as who carries the operational risk.

📚 Official docs

- OpenStack Architecture Design Guide
- OpenStack documentation

certs.eknathalabs.com

Q91 How do you design a highly available OpenStack control plane?

HA & scale

🎙 Spoken answer · ~56 s

I run at least three controller nodes so that clustered components keep quorum if one fails. In front of the stateless API services I place HAProxy with Keepalived providing a floating virtual IP, health-checking each backend. The database is a three-node MariaDB Galera cluster, with writes directed to one node through HAProxy to avoid certification conflicts. RabbitMQ runs as a cluster with quorum queues or mirrored queues, and clients know all nodes. Memcached is clustered through client-side hashing, and services such as Nova, Neutron and Glance APIs run on every controller. Where needed, Pacemaker manages ordering and fencing. Shared storage for Glance images uses Ceph. I test the design by killing each component and a whole node, and I watch for long recoveries, because HA that is never tested is a belief, not a feature.

Likely follow-ups

- Why direct writes to a single Galera node?
- How do quorum queues differ from mirrored queues?

Three controllers, HAProxy and Keepalived VIP, Galera for the DB, clustered RabbitMQ, memcached, shared image storage in Ceph. Test by killing components.

```
echo 'show stat' | socat stdio /var/lib/kolla/haproxy/haproxy.sock | cut -d, -f1,2,18
```

```
ip addr show | grep <vip>
```

```
rabbitmqctl list_queues name type
```

Interview tip: Say HA is only real if you have tested the failures.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q92 How do you operate a MariaDB Galera cluster for OpenStack, and what happens in a split brain?

HA & scale

🎙 Spoken answer · ~63 s

Galera is a synchronous multi-master cluster where each transaction is certified across all nodes using write sets. Quorum is based on majority, so a three-node cluster survives one failure, and a two-node cluster has no safe majority. OpenStack tolerates only a single writer well, because multi-writer certification conflicts cause deadlocks, so HAProxy sends writes to one primary node and keeps the others as backups. If the network partitions, the minority side goes non-primary and rejects queries, which prevents split brain; the majority continues. If all nodes stop, I must bootstrap from the node with the highest sequence number, found through the grastate file, rather than just starting all of them. I monitor wsrep cluster size, status, flow control and replication lag, and take regular backups with a tool such as mariabackup. If the interviewer pushes further, I would explain how I would test this in a staging environment before touching production.

Likely follow-ups

- How do you find the most advanced node?
- What is flow control?

Synchronous replication with majority quorum. Send writes to one node via HAProxy, and after a full outage bootstrap from the most advanced node using the sequence number. Monitor cluster size and flow control.

```
mysql -e "SHOW STATUS LIKE 'wsrep_%'" | grep -E
'cluster_size|cluster_status|local_state_comment'
```

```
cat /var/lib/mysql/grastate.dat
```

```
galera_new_cluster
```

Interview tip: Never start every node at once after a full outage; bootstrap the advanced one.

📚 Official docs

- Google SRE Book: Managing Critical State (consensus)
- OpenStack documentation
- Galera Cluster documentation

certs.eknathalabs.com

Q93 How do you keep RabbitMQ healthy at scale and recover from partitions?

HA & scale

🎙 **Spoken answer · ~57 s**

RabbitMQ is the nervous system, so I treat it carefully. I run a cluster of three nodes with quorum queues, which replace classic mirrored queues, because quorum queues use Raft and are safer under failures. I set a partition handling strategy, such as pause_minority, so a minority side stops rather than diverging. For clients, oslo.messaging heartbeats, retry intervals and timeouts are tuned, so that connections fail fast and reconnect. I separate noisy notifications, for example from Ceilometer, from RPC traffic, by using different exchanges or even a different broker, and I purge unconsumed notification queues, which otherwise grow until memory alarms fire. Monitoring covers queue depth, consumers per queue, memory, file descriptors and connection counts. After a partition or crash, sometimes the cleanest recovery is to restart the cluster and restart services so they re-declare queues.

Likely follow-ups

- What does pause_minority do?
- Why are notification queues dangerous?

Three-node cluster with quorum queues and pause_minority, tuned oslo.messaging heartbeats, separate or drain notification traffic, and monitor queue depth, memory and consumers.

```
rabbitmqctl cluster_status
```

```
rabbitmqctl list_queues name messages consumers | sort -k2 -nr | head
```

```
rabbitmqctl list_policies
```

Interview tip: Name notification queue buildup as the classic silent killer.

📚 **Official docs**

- RabbitMQ: Quorum queues
- RabbitMQ documentation
- oslo.messaging documentation

certs.eknathalabs.com

Q94 How do you scale an OpenStack cloud as it grows from tens to thousands of nodes?

HA & scale

🎙 Spoken answer · ~61 s

Scale has several bottlenecks and I address them in order. First, the message bus and database: use cells v2 to shard compute nodes, tune oslo.messaging, and put RabbitMQ per cell. Second, the API tier: add controllers, more workers, and caching. Third, Neutron: move from agent-based OVS to OVN, use routed provider networks, and reduce L2 domains. Fourth, storage: scale Ceph by adding OSD nodes and tune placement groups and networks. Fifth, observability and operations: automate everything, since manual steps do not scale, and standardise hardware and flavors. I also separate failure domains, with regions or multiple clouds if a single control plane becomes too large to upgrade safely. Throughout, I measure, load test with tools like Rally, and scale where the data shows contention instead of guessing. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- What is Rally used for?
- When do you add a region instead of scaling one?

Shard with cells, tune and split RabbitMQ and the DB, add API capacity, adopt OVN and routed networks, scale Ceph, automate operations, and measure with Rally before scaling.

```
rally task start boot-and-delete.yaml
```

```
nova-manage cell_v2 list_cells
```

```
ceph osd pool autoscale-status
```

Interview tip: Scale the measured bottleneck, and mention cells and OVN.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q95 What is Masakari and how does it add instance high availability?

HA & scale

🎙 Spoken answer · ~63 s

OpenStack itself does not restart instances automatically when a host dies. Masakari is the instance high-availability service that fills the gap. It runs monitors: host monitors detect compute node failure, process monitors check critical processes like nova-compute and libvirt, and instance monitors detect guests that crash. When it detects a failure, the Masakari engine executes a recovery workflow, for a failed host that is to evacuate the instances to a reserved or available host, for a crashed instance to restart it. Hosts are arranged in failover segments, and you can reserve hosts for recovery. It integrates with Pacemaker for host detection. You still need shared storage for fast recovery and you decide per-instance whether it should be recovered through metadata. Masakari addresses restart, not application-level HA, which still needs clustering above. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- What is a failover segment?
- Why do you still need application-level HA?

Masakari adds instance HA: host, process and instance monitors trigger recovery workflows such as evacuation or restart within failover segments. Pair it with shared storage and application-level clustering.

```
openstack segment list
```

```
openstack segment host list <segment>
```

```
openstack server set --property HA_Enabled=True web1
```

Interview tip: Say it restarts instances, not applications.

📚 Official docs

- Nova documentation
- Masakari documentation

certs.eknathalabs.com

Q96 How do you design Ceph failure domains and what happens when a rack fails?

HA & scale

🎙 Spoken answer · ~57 s

The CRUSH map describes the hierarchy of hosts, racks and rooms, and the replication rule says replicas must land in different failure domains, normally host at minimum and rack in larger clusters. With three replicas and a rack failure domain, each copy sits in a different rack, so losing a rack leaves two copies and the cluster keeps serving I/O, degraded. Ceph then re-replicates the missing data to the remaining racks, provided there is capacity and at least as many racks as replicas, which is why a three-rack cluster has no room to heal after one rack fails. I size so the rest can absorb the data, throttle backfill to protect client latency, and have monitoring for degraded and misplaced objects. For stretch designs over two sites, I would use stretch mode with a tiebreaker monitor.

Likely follow-ups

- Why do you need more racks than replicas?
- How do you throttle recovery?

CRUSH rules place replicas in separate hosts or racks. A rack loss leaves degraded but available data; healing needs spare capacity and enough racks. Throttle backfill and monitor degraded objects.

```
ceph osd tree
```

```
ceph osd crush rule dump
```

```
ceph config set osd osd_max_backfills 1
```

Interview tip: Say you need spare racks and capacity for self-healing.

📚 Official docs

- Ceph: Block devices and OpenStack
- Ceph documentation

certs.eknathalabs.com

Q97 How do you plan disaster recovery for an OpenStack cloud?

HA & scale

🎙 Spoken answer · ~65 s

I separate the recovery of the platform from the recovery of tenant workloads. For the platform, I back up the databases regularly and test restores, keep the deployment repository and secrets safe, and be able to rebuild controllers quickly from automation. The configuration, Fernet keys and certificates need backup too. For tenant data, I offer options: Cinder backups to another site, volume replication, Ceph RBD mirroring between clusters, Glance image replication, and application-level replication. A second region or cloud provides a failover target, with documented RTO and RPO for each tier. I also address networking, DNS and floating IP failover. Most importantly, I run game days, where a site failure is simulated and the process is timed, because real DR depends on practised steps, and I record what was missed. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- What is RBD mirroring?
- What belongs in the platform backup?

Back up the platform (DB, keys, config, deploy repo) and offer tenants backups, replication and RBD mirroring across sites, with defined RTO and RPO. Practice with game days.

```
mariabackup --backup --target-dir=/backup/$(date +%F)
```

```
rbd mirror pool status volumes
```

```
openstack volume backup list
```

Interview tip: Say you separate platform DR from tenant DR.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q98 How do you run a multi-region OpenStack with shared Keystone?

HA & scale

🎙 Spoken answer · ~58 s

A region is a full deployment with its own Nova, Neutron, Cinder and Glance, sharing identity so users have one login. The simplest pattern is a shared Keystone, either a single highly available instance in one region with a database replicated to the others, or a Keystone cluster spanning sites. Each service endpoint in the catalog is tagged with its region, so clients choose the region with the region name option. Alternatively, each region runs its own Keystone and they are tied by federation, which reduces the shared failure risk at the cost of more complexity. Images and quotas are managed per region, and cross-region workflows need explicit tooling, for example image replication or Heat with multiple regions. The main risks are Keystone latency and availability, so I cache aggressively and design identity to survive a region outage.

Likely follow-ups

- What is the risk of a single Keystone?
- How do users select a region?

Regions share identity through a replicated or clustered Keystone, or federate separate ones. The catalog marks endpoints by region; cross-region workflows need tooling and Keystone is the risk to design around.

```
openstack region list
```

```
openstack --os-region-name RegionTwo server list
```

```
openstack catalog list
```

Interview tip: Name Keystone as the shared fate to design around.

📚 Official docs

- OpenStack Architecture Design Guide
- OpenStack documentation
- Keystone documentation

certs.eknathalabs.com

Q99 How do you prevent and handle a thundering herd of API requests on the control plane?

HA & scale

🎙 Spoken answer · ~58 s

A burst of requests, for example thousands of instances booting at once after a power event, can overload the API, queue and database. I protect the control plane at several levels. At the edge, HAProxy or the API gateway applies rate limits and connection caps. Inside services, I tune worker counts, database connection pools and RPC timeouts to prevent cascading failure. Throttling parameters such as max_concurrent_builds on compute nodes, and image download concurrency, limit local overload. For recovery after outages, I bring services up in a controlled order, and I use start-delay on instances with resume_guests_state_on_host_boot to avoid all guests starting simultaneously. For tenants, quotas and API rate limits reduce abusive scripts, and I watch queue depth and latency percentiles to see saturation early. Load tests such as Rally verify the limits before they are hit in production.

Likely follow-ups

- What does max_concurrent_builds do?
- How do you restart a whole cloud in order?

Rate limit at HAProxy, tune workers and pools, cap concurrent builds, stage restarts, set quotas and watch queue depth and latency percentiles. Validate with load tests.

```
crudini --set /etc/nova/nova.conf DEFAULT max_concurrent_builds 5
```

```
echo 'show info' | socat stdio /var/lib/kolla/haproxy/haproxy.sock | grep -i conn
```

```
rally task start boot-storm.yaml
```

Interview tip: Mention controlled restart order after a power event.

📚 Official docs

- Google SRE Book: Addressing Cascading Failures
- Swift documentation

certs.eknathalabs.com

Q100 How do you perform a rolling upgrade of OpenStack with minimal downtime? Operations

🎙 Spoken answer · ~58 s

I treat an upgrade as a project with rehearsal. First I read the release notes for removed options and database migrations, back up the databases and configuration, and run the whole upgrade in staging. In production, I upgrade in a defined order. Control-plane services go first, with expand schema migrations run online, services restarted one controller at a time behind HAProxy, using oslo.versionedobjects so new and old versions can coexist. Then the compute nodes are upgraded in batches after draining or accepting a brief agent restart, since running guests are not affected by restarting nova-compute. Finally I run the contract migrations and cleanup, and remove deprecated config. Throughout, I run smoke tests and canaries, watch error rates, keep rollback to previous images or packages ready, and avoid mixing releases longer than necessary. SLURP lets me skip intermediate releases.

Likely follow-ups

- Why can old and new services coexist?
- What is the expand and contract migration pattern?

Rehearse in staging, back up, upgrade the control plane one node at a time, then computes in batches, run contract migrations, and canary throughout. Versioned objects allow temporary mixed versions.

```
kolla-ansible -i multinode upgrade
```

```
nova-status upgrade check
```

```
nova-manage db online_data_migrations
```

Interview tip: Mention upgrade checks and online data migrations by name.

📚 Official docs

- Nova: Upgrades
- OpenStack documentation

certs.eknathalabs.com

Q101 How do you monitor OpenStack: which metrics and alerts matter most?

Operations

🎙 Spoken answer · ~67 s

I monitor in layers. At the infrastructure layer, host CPU, memory, disk, network and hardware health through node exporter and IPMI. At the platform layer, service state: nova-compute, neutron agents and cinder-volume heartbeat, API response codes and latency per endpoint, RabbitMQ queue depth and consumers, Galera cluster size and flow control, HAProxy backends, and Ceph health, slow ops and capacity. At the capacity layer, hypervisor vCPU and RAM allocation versus overcommit, Placement usage, floating IP and fixed IP pool consumption, and quotas near limits. At the tenant experience layer, a synthetic canary that boots a VM and pings it, with success rate and duration, which I consider the single most valuable alert. I use Prometheus with the OpenStack exporter and Grafana, alert on symptoms first, and write a runbook link into every alert. I would finish by naming the one log line or metric that proves the change worked, because verification is what separates a confident answer from a guess.

Likely follow-ups

- Why is the canary so valuable?
- Which alert would page the on-call?

Layered monitoring: hosts, service heartbeats and API latency, RabbitMQ, Galera and Ceph health, capacity versus allocation, and a synthetic boot-and-ping canary that mirrors the tenant experience.

```
openstack-exporter --os-client-config /etc/openstack/clouds.yaml
```

```
curl -s localhost:9180/metrics | grep openstack_nova_up
```

```
promtool check rules openstack.rules.yml
```

Interview tip: The synthetic canary is the alert that reflects users.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q102 What are Ceilometer, Gnocchi and Aodh, and how do they relate to billing and autoscaling?

Operations

🎙 **Spoken answer · ~65 s**

Ceilometer collects telemetry, meaning samples and events from OpenStack services and instances, such as CPU time, network bytes and disk use. Gnocchi is a time-series database designed to store those measures efficiently, aggregated by resource and archive policy. Aodh evaluates alarm rules on the metrics or events and triggers actions like webhooks or Heat scaling policies. CloudKitty then rates the usage into billing or showback data. Together they power tenant autoscaling, usage reports and chargeback. However, in many modern clouds the operational monitoring has moved to Prometheus, and Ceilometer plus Gnocchi is used where billing or Heat autoscaling is needed, because Gnocchi had scaling and maintenance challenges in large deployments. When asked, I explain what each does and note that you can choose a lighter path depending on the requirement. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- Why do many operators prefer Prometheus now?
- What does CloudKitty do?

Ceilometer collects, Gnocchi stores time series, Aodh alarms and triggers, CloudKitty rates usage. They support billing and Heat autoscaling; Prometheus often replaces them for operational monitoring.

```
openstack metric resource list
```

```
openstack alarm create --name cpu-high --type gnocchi_resources_threshold --metric cpu --
threshold 70000000000 --resource-type instance --resource-id <id>
```

```
openstack rating summary get
```

Interview tip: Separate operational monitoring from usage metering.

📚 **Official docs**

- Ceilometer documentation

certs.eknathalabs.com

Q103 How do you handle capacity planning and chargeback or showback?

Operations

🎙 Spoken answer · ~65 s

Capacity planning compares supply to demand with forecasts. I track allocation, meaning what is promised through flavors and quotas, and utilisation, meaning what is actually used, for vCPU, RAM, disk and network, per aggregate and per availability zone. I use trends and growth projects from onboarding pipelines to estimate when I will cross thresholds, typically setting a trigger at 70 to 80 percent allocation, and I factor in lead time for hardware procurement, which can be months. I also keep failure headroom. For chargeback or showback, I use CloudKitty or exported usage to attribute cost per project, with unit prices by flavor, volume type and floating IP. Showing teams their cost changes behaviour even without invoices, by encouraging shutdown of idle instances. Regular reviews with tenants tune quotas and sizes to real needs. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- Why track both allocation and utilisation?
- What lead time should drive thresholds?

Track allocation and utilisation per aggregate, forecast growth against procurement lead time, keep failure headroom, and use CloudKitty or exports for showback to change tenant behaviour.

```
openstack hypervisor stats show
```

```
openstack usage show --project payments
```

```
openstack rating dataframes get
```

Interview tip: Say hardware lead time sets your capacity thresholds.

📚 Official docs

- Google SRE Book: Software Engineering in SRE (capacity planning)
- OpenStack Operations Guide
- CloudKitty documentation

certs.eknathalabs.com

Q104 How do you patch the OS and kernel on hosts in a running cloud?

Operations

🎙 Spoken answer · ~59 s

I plan patching as a rolling operation across failure domains. For compute nodes, I drain one host at a time or one rack at a time, live-migrating instances away, then patch, reboot, verify services, and return it to service, using automation that checks health before moving on. For controllers, I patch one at a time, confirming that the cluster is healthy, Galera synced, RabbitMQ rejoined, and the VIP stable, before the next. Storage nodes are done carefully, one failure domain at a time, setting noout in Ceph so a reboot does not trigger data movement, and waiting for the cluster to return to health. Kernel and hypervisor updates for security fixes get higher priority, and I track the CVE exposure. I keep a patch calendar, test on staging first, and report compliance, because unpatched hosts become a compliance and security problem.

Likely follow-ups

- Why set noout before rebooting an OSD host?
- What do you check between controller reboots?

Patch one failure domain at a time with health gates: drain and migrate computes, patch controllers sequentially with Galera, RabbitMQ and VIP checks, and use Ceph noout for storage reboots.

```
ceph osd set noout
```

```
kolla-ansible -i multinode stop --limit compute05
```

```
needs-restarting -r
```

Interview tip: Mention health gates between each node.

📚 Official docs

- OpenStack Operations Guide

certs.eknathalabs.com

Q105 How do you tune the performance of a busy OpenStack cloud?

Operations

🎙 Spoken answer · ~57 s

I measure first, then tune the biggest bottleneck. On the control plane, I look at API latency per service, database slow queries and connection counts, RabbitMQ queue depth, and worker utilisation. Typical fixes are more API and conductor workers sized to CPU, tuned DB connection pools, indexes or archiving old rows, which is the purpose of nova-manage db archive_deleted_rows and purge, moving to OVN to cut agent chatter, and caching with memcached. On compute, I tune CPU governors, NUMA and pinning for sensitive workloads, hugepages, virtio multiqueue, and disk cache modes. In networking, I enable jumbo frames, offloads, and tune conntrack tables. In storage, I look at Ceph latency, OSD CPU, network speed, and use separate pools for different tiers. Every change is made one at a time, measured with Rally or application tests, and recorded.

Likely follow-ups

- Why archive deleted rows?
- What does virtio multiqueue improve?

Measure, find the bottleneck, then tune workers, DB pools and archiving, memcached, OVN, NUMA and hugepages, jumbo frames and Ceph tiers, one change at a time with before and after tests.

```
nova-manage db archive_deleted_rows --until-complete --max_rows 10000
```

```
mysqldumpslow /var/log/mysql/slow.log | head
```

```
sysctl net.netfilter.nf_conntrack_max
```

Interview tip: One change at a time, measured, is the discipline to highlight.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q106 How do you onboard a new tenant or team into the cloud consistently?

Operations

🎙 Spoken answer · ~58 s

I make it a self-service or automated workflow, so every tenant starts with the same secure baseline. A request, from a ticket or a portal, triggers automation that creates the project in the right domain, creates groups mapped to the identity provider, assigns roles, sets quotas by size, creates a default network with a router and subnet, a default security group with sensible rules, and keys and tags for ownership and cost centre. Notifications go to the team with documentation and endpoints. Naming standards and tags make reporting possible. I also set policy for who can create public networks and floating IPs, and for image sources. The automation is code in Git, idempotent, and every step is logged for audit. Later, offboarding follows the reverse process, snapshotting if needed and freeing resources, so abandoned projects do not consume capacity.

Likely follow-ups

- Why map groups from the IdP?
- How do you handle offboarding?

Automate project creation, role assignment via IdP groups, quotas, default network and security group, and tags for owner and cost centre, with logging and a matching offboarding flow.

```
ansible-playbook onboard_tenant.yml -e team=payments size=medium
```

```
openstack project show payments -c tags
```

```
openstack quota show payments
```

Interview tip: Mention offboarding to show lifecycle thinking.

📚 Start here (official docs)

- OpenStack documentation

certs.eknathalabs.com

Q107 How do you manage logs centrally and use them to troubleshoot OpenStack?

Operations

🎙 Spoken answer · ~57 s

I ship logs from every node and container to a central store such as Elasticsearch or OpenSearch, or Loki, using an agent like Fluentd, Filebeat or Promtail. Parsing extracts fields like service, level, request ID, instance UUID and project, so I can search across components. During an incident I start with the request ID from a failing API call and follow it through Nova, Neutron, Cinder and Glance in one query, which turns a multi-service puzzle into a timeline. I set retention by value, longer for audit and security logs, shorter for debug, and I keep debug off by default because it floods storage and may log sensitive data. Dashboards show error rates per service, and alerts trigger on patterns such as repeated MessagingTimeout or 'No valid host'. Time synchronisation across nodes is essential for readable timelines.

Likely follow-ups

- Why must clocks be synchronised?
- What should alerts match in logs?

Central log store with parsed request ID, instance and project fields; follow a request ID across services; alert on MessagingTimeout and No valid host patterns; keep debug off by default.

```
kubectl logs -n openstack deploy/nova-api | grep req-
```

```
curl -s 'localhost:9200/_search?q=req-abc123'
```

```
journalctl -u docker --since '10 min ago' | grep ERROR
```

Interview tip: Request ID correlation is the standout technique.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q108 How do you clean up orphaned resources and stale data in OpenStack?

Operations

🎙 Spoken answer · ~67 s

Clouds accumulate orphans: volumes and ports of deleted instances, floating IPs not associated, stale images, deleted rows in the database and abandoned projects. I tackle it with scheduled reports and safe automation. Reports list unattached volumes older than a threshold, down ports, unused floating IPs, instances in ERROR for days, and projects with no activity. Owners are notified first, with a grace period, and resources are tagged. Cleanup of database soft-deleted rows uses archive and purge commands to keep tables small. For leaked or inconsistent resources between services, I use the documented tools, such as the Neutron or Nova database consistency checks, rather than manual SQL. Every cleanup is logged and reversible where possible, for example by snapshotting volumes before deleting. The goal is cost control, performance and security hygiene without surprising tenants. I would finish by naming the one log line or metric that proves the change worked, because verification is what separates a confident answer from a guess.

Likely follow-ups

- How do you avoid deleting something still in use?
- What does purge do?

Scheduled reports of unattached volumes, idle IPs, error instances and dormant projects, notify owners with a grace period, archive and purge DB rows, and use official consistency tools.

```
openstack volume list --all-projects --status available
```

```
openstack floating ip list --status DOWN
```

```
nova-manage db purge --all-cells --before 2026-01-01
```

Interview tip: Notify owners and keep it reversible.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q109 How do you manage image lifecycles and golden images?

Operations

🎙 Spoken answer · ~63 s

I build images through a pipeline instead of by hand: Packer or diskimage-builder takes a base distribution, applies configuration, installs cloud-init, agents and hardening, runs tests, and uploads the result to Glance with properties and tags. Images are rebuilt on a schedule, for example monthly, and when critical vulnerabilities appear, and each build is versioned by date and recorded in a manifest. Old images are deprecated through visibility settings and tags, and eventually removed after tenants have moved. I mark which images are supported, and set required properties such as hw_disk_bus or os_distro for correct virtual hardware. Signing images and validating signatures in Nova stops tampered images from booting. Documentation tells tenants how to find the latest image, and an alert flags instances running an image that is out of support. The key point I want the interviewer to hear is that I understand the trade-off, not just the definition.

Likely follow-ups

- How do you deprecate an image safely?
- What does image signature verification protect against?

Build with Packer or diskimage-builder in a tested pipeline, version and rebuild monthly and on CVEs, deprecate via visibility, sign and verify, and flag instances running old images.

```
diskimage-create -o ubuntu.qcow2 ubuntu-minimal
```

```
openstack image set --property os_distro=ubuntu --tag supported ubuntu-2026-10
```

```
openstack image set --private ubuntu-2025-04
```

Interview tip: Pipelines, signing and deprecation are the three points.

📚 Official docs

- Packer documentation
- Glance documentation

certs.eknathalabs.com

Q110 How do you restore service after losing the whole database?

Operations

🎙 Spoken answer · ~61 s

I stay calm and follow a tested runbook. First, stop the OpenStack services that write to the database, so nothing further changes while I restore. Then I restore the most recent consistent backup, for a Galera cluster that usually means restoring on one node, bootstrapping the cluster from it, and letting the other nodes rejoin through state transfer. If I also have binary logs, I can replay to a point in time and reduce data loss. After the database is back, I start services and then reconcile reality with records: instances that were created after the backup may exist on hypervisors but not in the database, and instances deleted later may reappear. I compare libvirt domains, Ceph images and ports with the restored records, clean up or re-adopt orphans, and tell tenants about any lost changes. The lesson is frequent backups, tested restores and a documented RPO.

Likely follow-ups

- How do you handle drift between hypervisors and the restored DB?
- What is point-in-time recovery?

Stop writers, restore the last consistent backup, bootstrap Galera from it, apply binlogs if available, then reconcile hypervisor, Ceph and port reality against the restored records.

```
mariabackup --prepare --target-dir=/backup/2026-10-20
```

```
mariabackup --copy-back --target-dir=/backup/2026-10-20
```

```
virsh list --all
```

Interview tip: Reconciliation after restore is the step people forget.

📚 Official docs

- PostgreSQL: Continuous archiving and PITR
- Galera Cluster documentation

certs.eknathalabs.com

Q111 What does a good change-management and maintenance-window process look like for a cloud?

Operations

🎙 **Spoken answer · ~56 s**

Every change to the platform goes through a lightweight but real process. A change request describes what, why, the risk, the rollback and the test plan, and is peer reviewed. Changes are implemented from code in Git through the deployment tool, first in staging and then production, in a published maintenance window with notice to tenants, scaled to impact. During the window I follow a checklist with validation steps, keep a live log, and have a clear abort point at which I roll back. Afterwards, a short review records the outcome and updates runbooks. For emergency changes there is an expedited path, with retrospective review. Good communication matters as much as the technical steps: a status page, advance notices and clear closure messages build the trust that makes maintenance easier to schedule next time.

Likely follow-ups

- What defines the abort point?
- How do emergency changes differ?

Reviewed change requests with risk, test and rollback, applied from Git via the deployment tool, staged first, with tenant notice, a checklist and an abort point, then a short review.

```
git tag change-2026-10-21 && git push --tags
```

```
kolla-ansible -i multinode deploy --check
```

```
openstack compute service list
```

Interview tip: Communication is as important as the technical steps.

📚 **Official docs**

- OpenStack Operations Guide

certs.eknathalabs.com

Q112 How do you keep OpenStack documentation and runbooks useful and current?

Operations

🎙 **Spoken answer · ~64 s**

Runbooks decay quickly unless they are part of the workflow. I keep them in Git next to the automation, written as short, step-by-step procedures with prerequisites, commands, expected output, and rollback, and I link them from alerts so the on-call engineer lands on the right page. Where possible, I turn a runbook into code, such as an Ansible playbook, so the doc describes the script instead of replacing it. Every incident ends with a review that includes updating or creating a runbook. I schedule quarterly walk-throughs where someone who did not write the runbook follows it in staging, and fix every unclear step. Architecture diagrams, IP plans and decision records are versioned alongside. The measure of success is that a new on-call engineer can resolve a common failure using the document alone. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- How do you test a runbook?
- What belongs in an architecture decision record?

Runbooks live in Git, are linked from alerts, are turned into playbooks where possible, updated after every incident, and tested by someone who did not write them.

```
git log --stat -- runbooks/
```

```
grep -R 'runbook_url' alerts/
```

```
ansible-playbook runbooks/restart_rabbitmq.yml --check
```

Interview tip: Tested-by-a-newcomer is the best quality signal.

📚 **Official docs**

- OpenStack Operations Guide
- OpenStack documentation

certs.eknathalabs.com

Q113 How do you harden an OpenStack deployment?

Security

🎙 Spoken answer · ~60 s

I work through layers. Identity: enforce secure RBAC, MFA through federation, application credentials for automation, and short token lifetimes. Network: isolate management, storage and tenant networks, restrict the API endpoints with firewalls, TLS for all endpoints including internal, and no direct database or queue exposure. Hosts: minimal OS, regular patches, SELinux or AppArmor enforcing, hardened SSH with keys only, CIS benchmarks, and disabled unused services. Services: run as non-root, containers with limited capabilities, secrets in a vault and encrypted at rest, and sensible default security groups. Data: volume encryption with Barbican, backups encrypted. Operations: central audit logging, intrusion detection, quotas, rate limits, and a vulnerability management process following OpenStack security advisories. I validate with periodic reviews against the OpenStack Security Guide. Finally, I would say how I would document this for the team so the knowledge does not live only in one person's head.

Likely follow-ups

- Which standard would you benchmark against?
- How do you monitor security advisories?

Secure RBAC and MFA, network isolation and TLS everywhere, hardened hosts with SELinux, secrets in a vault, encrypted volumes, audit logs and a patching process aligned with the Security Guide.

```
getenforce
```

```
openssl s_client -connect <vip>:5000 </dev/null | grep -i verify
```

```
oscap xccdf eval --profile cis /usr/share/xml/scap/ssg-ubuntu2204-ds.xml
```

Interview tip: Offer a layered list, then name the Security Guide.

📚 Official docs

- OpenStack Security Guide
- OpenStack documentation

certs.eknathalabs.com

Q114 What is Barbican and how does it manage secrets and keys?

Security

🎙 Spoken answer · ~57 s

Barbican is the OpenStack key manager. It stores and manages secrets such as passwords, certificates, symmetric keys and private keys, with access control through Keystone. A secret has a payload, metadata and an optional expiry, and is stored encrypted through a plugin back end: the simple crypto plugin using a master key, a PKCS#11 hardware security module, or Vault integration. Other services use it: Cinder and Nova for encrypted volumes, Octavia for TLS listener certificates, Glance for image signing and Magnum for cluster certificates. Containers group related secrets, for example a certificate with its key. Access is controlled by project and role, and ACLs can share a secret with specific users. Because it holds the keys to many other protections, Barbican itself needs to be highly available, backed up, and protected with an HSM for sensitive environments.

Likely follow-ups

- Which services depend on Barbican?
- Why consider an HSM back end?

Barbican stores secrets and keys with Keystone access control and pluggable encrypted back ends. It supports encrypted volumes, Octavia TLS, image signing and Magnum certificates.

```
openstack secret store --name db-pass --payload 's3cret'
```

```
openstack secret list
```

```
openstack secret container create --type certificate --secret certificate=<ref> --secret
private_key=<ref> --name cert1
```

Interview tip: Say it holds keys for many services, so it must be HA and backed up.

📚 Official docs

- Barbican documentation

certs.eknathalabs.com

Q115 How do you isolate tenants from each other in OpenStack?

Security

🎙 Spoken answer · ~65 s

Isolation exists at several layers. Compute: each instance runs in its own KVM guest with hardware virtualisation, with SELinux sVirt or AppArmor confining the QEMU process, so a guest escape is contained. Network: tenant networks are separate segments, VXLAN, Geneve or VLAN, with security groups and port security preventing spoofing, and private routers so overlapping address ranges are isolated. Storage: volumes are owned by projects and access is by project scope, with optional encryption per volume. Identity: Keystone projects and domains with RBAC. For stronger isolation, I use dedicated host aggregates so sensitive tenants do not share hypervisors, disable memory deduplication features, and sometimes use bare metal through Ironic. Quotas and QoS reduce noisy-neighbour effects. Isolation is a combination of defaults and careful configuration, not a single switch. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- What does sVirt do?
- When would you dedicate hosts to a tenant?

KVM guests confined by sVirt or AppArmor, separate network segments with security groups and port security, project-scoped storage and RBAC, and dedicated aggregates or bare metal for the most sensitive tenants.

```
ps -eZ | grep qemu
```

```
openstack port show <port> -c port_security_enabled
```

```
openstack aggregate create --property tenant=secure secure-hosts
```

Interview tip: Say isolation is layered: compute, network, storage, identity.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q116 How do you audit activity in OpenStack?

Security

🎙 Spoken answer · ~64 s

Auditing needs a record of who did what, when and from where. I enable the audit middleware, based on pycadf, which emits CADF-format events for API calls to the message bus or log, including user, project, action and outcome. I send those logs, plus Keystone authentication events, system auth logs and host audit logs from auditd, to a central write-once store or SIEM, with retention set by policy. Alerts cover privilege changes, role assignments, failed logins, creation of application credentials, and API calls from unusual locations. Time synchronisation keeps timelines accurate, and request IDs tie operations together. I also make sure administrators use named accounts, never a shared admin, so accountability is possible. Periodic access reviews check who holds elevated roles. Protecting the audit logs from tampering is part of the design. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- What is CADF?
- Why avoid a shared admin account?

Enable audit middleware for CADF events, centralise API, Keystone, auditd and auth logs in a tamper-resistant store, alert on role changes and failed logins, and use named admin accounts.

```
grep -i 'audit' /etc/nova/api-paste.ini
```

```
ausearch -m USER_LOGIN -ts today
```

```
openstack role assignment list --names --role admin
```

Interview tip: Named accounts and tamper-resistant storage are the key points.

📚 Official docs

- OpenStack Security Guide
- OpenStack documentation

certs.eknathalabs.com

Q117 How do you respond to an OpenStack security advisory (OSSA) or a critical CVE?

Security

🎙 Spoken answer · ~66 s

I follow a defined process. First, triage: read the advisory, identify the affected services and versions, and check whether my deployment is exposed, which depends on configuration and network reachability. Second, assess severity and exploitability, for example whether it requires authentication, and decide a timeline. Third, find the fix: a backported patch in my release branch, an updated container image or distribution package, or a configuration mitigation if the patch is not ready. Fourth, test in staging, then roll out through the normal deployment tool, in batches, with monitoring. Fifth, verify that the vulnerable version is gone and record the change. For critical issues, I accelerate through an emergency change path and consider temporary controls like firewall rules. Subscribing to the OpenStack security announcements list and my vendor's feed keeps me informed. I would also mention the main failure mode and how I would detect it early, since operations experience shows in how you talk about what goes wrong.

Likely follow-ups

- How do you mitigate before a patch exists?
- How do you verify the fix is deployed everywhere?

Triage exposure, assess severity, get the backported fix or a mitigation, test in staging, roll out in batches, verify the fixed version everywhere, and document. Use an emergency path for critical issues.

```
docker ps --format '{{.Image}}' | sort -u | grep nova
```

```
pip show nova | grep Version
```

```
kolla-ansible -i multinode deploy --limit control
```

Interview tip: Say exposure assessment comes before panic patching.

📚 Official docs

- Trivy
- OpenStack Security Guide
- OpenStack documentation

certs.eknathalabs.com

Q118 How do you secure the hypervisor and the compute host?

Security

🎙 Spoken answer · ~64 s

The compute host is a high-value target because it holds many tenants. I install a minimal OS, keep it patched, and allow only required ports, with management access through a bastion or jump host. SELinux in enforcing mode with sVirt gives each QEMU process a unique label, so one guest cannot access another's files. I disable unnecessary services, restrict who can use libvirt, and protect the libvirt and migration channels with TLS or SSH. Kernel hardening and mitigations for CPU side-channel vulnerabilities are applied, with awareness of performance effects, and for the highest isolation, I disable simultaneous multithreading or schedule tenants on separate hosts. Secure boot, a TPM and measured boot help detect tampering. The BMC network is isolated since BMC compromise bypasses the OS. Regular compliance scans confirm the configuration remains hardened. If the interviewer pushes further, I would explain how I would test this in a staging environment before touching production.

Likely follow-ups

- Why isolate the BMC network?
- What are CPU side-channel mitigations?

Minimal patched OS, SELinux sVirt, libvirt access controls and TLS, kernel and side-channel mitigations, secure boot, isolated BMC network and regular compliance scans.

```
getsebool -a | grep virt
```

```
virsh -c qemu:///system list
```

```
cat /sys/devices/system/cpu/vulnerabilities/*
```

Interview tip: Mention the BMC network; it is often overlooked.

📚 Official docs

- OpenStack Nova
- OpenStack Security Guide

certs.eknathalabs.com

Q119 How do you protect the OpenStack APIs from abuse and denial of service?

Security

🎙 **Spoken answer · ~58 s**

I combine edge and internal controls. At the edge, a load balancer or WAF applies rate limits per client IP, connection and request size limits, and blocks malformed requests, with public API endpoints reachable only where needed, sometimes only from corporate networks or a VPN. Inside, Keystone and service middleware can throttle, and quotas limit what one tenant can create. I monitor for spikes in authentication failures and API errors, and set alerts for unusual request rates by project. I separate public and internal endpoints, so internal service traffic is not affected by public load. Worker and queue limits prevent one expensive call type from starving others, and long operations are asynchronous. For incidents, I can quickly block an abusive source at the firewall and disable a compromised application credential. Testing with load tools shows where the limits actually are.

Likely follow-ups

- How do quotas help with API abuse?
- How do you block a compromised credential quickly?

Rate limit and cap connections at HAProxy or a WAF, restrict public exposure, use quotas and middleware throttling, separate internal and public endpoints, and alert on auth failure spikes.

```
echo 'show table' | socat stdio /var/lib/kolla/haproxy/haproxy.sock
```

```
openstack application credential delete <id>
```

```
grep -c ' 401 ' /var/log/haproxy.log
```

Interview tip: Separate public and internal endpoints to contain load.

📚 **Official docs**

- OpenStack documentation

certs.eknathalabs.com

Q120 How do you handle secrets inside instances and automation without leaking them?

Security

🎙 Spoken answer · ~57 s

Never bake secrets into images or user data in plain text, since user data is readable from the metadata service by anything in the guest and by project members through the API. Instead I deliver secrets at runtime from a secret manager, such as Barbican or Vault, and let the instance prove its identity, for example through a short-lived token or an approle bound to the instance metadata. For automation, I use application credentials with narrow roles and an expiry, stored in the CI secret store, never in the repository, and clouds.yaml files with restricted permissions. I rotate credentials regularly and after staff changes, and scan repositories for leaked secrets. Logs must not print secrets, so debug logging is disabled in production. Finally, I assume leaks will happen and design for fast revocation and small blast radius.

Likely follow-ups

- Why is user data not a safe place for secrets?
- How can an instance authenticate to Vault?

Deliver secrets at runtime from Barbican or Vault with short-lived identity, avoid secrets in images or user data, use scoped expiring application credentials, rotate, scan repos and plan for revocation.

```
vault write auth/approle/role/app token_ttl=15m
```

```
chmod 600 ~/.config/openstack/clouds.yaml
```

```
git secrets --scan
```

Interview tip: Remember user data is readable; do not put secrets there.

📚 Official docs

- Nova documentation

certs.eknathalabs.com

Q121 Boot fails with 'No valid host was found'. How do you investigate?

Troubleshooting

🎙 Spoken answer · ~63 s

This error means the scheduler could not find any host that passed all steps, and the useful information is in the details, not the message. First I check the instance fault and the nova-scheduler log by request ID, which lists how many hosts each filter rejected. If Placement returned no candidates, the issue is raw capacity, requested resources or traits, so I look at the flavor, resource provider inventories and allocations, and at disabled or down compute services. If a filter removed everything, I examine that rule: the availability zone, an aggregate property, anti-affinity in a server group, a missing image property, NUMA or PCI requests, or a host that has no free pinned CPUs. I also check that nova-compute is reporting and that the hypervisor list shows resources. Once I know the cause I fix it, for example by adding capacity, relaxing a hint, or correcting metadata, and retry.

Likely follow-ups

- What if Placement shows free capacity but the build fails?
- How do traits cause this error?

Read the scheduler log by request ID for per-filter rejection counts; check Placement inventories and allocations, disabled computes, AZ, aggregate, server group, NUMA or PCI requirements, then fix the specific cause.

```
openstack server show web1 -c fault
```

```
grep <req-id> /var/log/nova/nova-scheduler.log
```

```
openstack allocation candidate list --resource VCPU=4 --resource MEMORY_MB=8192
```

Interview tip: Say the per-filter counts in the scheduler log tell you where it failed.

📚 Official docs

- Nova: Scheduling and filters

certs.eknathalabs.com

Q122 A floating IP is associated but the instance is unreachable from outside. What do you check?

Troubleshooting

🎙 Spoken answer · ~62 s

I test layer by layer. First, the instance itself: is it running, does it have its private address, and is its security group allowing the traffic, ICMP or SSH, from the source I am using? Second, the router: is the external gateway set, is the floating IP's NAT rule present in the router namespace, and can the router reach the instance subnet? Third, the external network: does the upstream router know how to reach the floating IP range, through ARP on a shared segment or a route or BGP advertisement, and is the router's gateway port up with its MAC visible on the physical network? Fourth, DVR or OVN specifics, such as external connectivity on the right host, or a gateway chassis that is down. I use tcpdump on the external bridge to see whether packets arrive and whether replies leave, which tells me which side is dropping them.

Likely follow-ups

- How do you verify the NAT rule exists?
- What if the upstream ARP entry is stale?

Check the instance and security group, the router namespace NAT and gateway, then the upstream route or ARP for the floating range, and capture on the external bridge to see which side drops.

```
openstack floating ip show 203.0.113.10
```

```
ip netns exec qrouter-<id> iptables -t nat -S | grep 203.0.113.10
```

```
tcpdump -ni br-ex host 203.0.113.10
```

Interview tip: The packet capture on the external bridge splits the problem in half.

📚 Official docs

- Neutron: Self-service networks
- Nova documentation

certs.eknathalabs.com

Q123 The nova-compute service shows as down but the host is running. How do you troubleshoot?

Troubleshooting

🎙 **Spoken answer · ~63 s**

Nova marks a service down when it has not updated its heartbeat in the database within a time limit, so I find out why the heartbeat stopped. I check that the process or container is actually running and not hung, and read its log for errors, such as lost connection to RabbitMQ, database connection errors or libvirt failures. I verify time sync, since heavy clock skew affects heartbeat evaluation. I test connectivity from the host to the message queue and database, and check CPU and memory exhaustion on the host. A common cause is a broken RabbitMQ connection that does not recover, where restarting nova-compute fixes it, but I still look for the underlying instability. If libvirt is stuck or its socket is unresponsive, instances may be fine but management is not, so restart libvirt carefully. Afterwards I confirm the service reports up, and the scheduler can use the host.

Likely follow-ups

- What is the heartbeat interval and timeout?
- Why does the time skew matter?

Check the process, its log for RabbitMQ, DB or libvirt errors, time sync, resource exhaustion and connectivity to queue and DB; restart if needed and look for the underlying instability.

```
openstack compute service list --service nova-compute
```

```
docker logs --tail 100 nova_compute
```

```
virsh list
```

Interview tip: Restarting hides the cause; look for why the heartbeat stopped.

📚 **Official docs**

- OpenStack Nova
- OpenStack documentation

certs.eknathalabs.com

Q124 Live migration fails or hangs. How do you debug it?

Troubleshooting

🎙 Spoken answer · ~60 s

I gather the migration status first, with the server migration list and the nova-compute logs on both the source and destination hosts, using the request ID. Then I check the usual causes in order. Compatibility: the destination must have a compatible CPU model and the same libvirt and QEMU features, otherwise the error mentions CPU or machine type. Connectivity: libvirt migration ports, SSH or TLS between hosts, and name resolution. Storage: both hosts need access to the same volumes or shared backend, or block migration must be selected. Networking: Neutron port binding for the destination, and security group or OVS setup. Capacity: enough free resources on the target. If the migration runs but never completes, the guest is dirtying memory faster than it copies, so I enable auto-converge or post-copy, or raise the permitted downtime. I then abort cleanly if needed and retry.

Likely follow-ups

- How do you abort a stuck migration?
- How do you make CPU models consistent?

Check migration status and logs on both hosts, then CPU compatibility, libvirt connectivity, storage access, port binding and capacity. For non-converging guests use auto-converge or post-copy.

```
openstack server migration list --server web1
```

```
nova live-migration-abort web1 <migration-id>
```

```
virsh domjobinfo <instance>
```

Interview tip: Name auto-converge and post-copy as the fix for busy guests.

📚 Official docs

- OpenStack Nova
- Nova: Live migration

certs.eknathalabs.com

Q125 RabbitMQ queues are growing and operations are slow or timing out. What do you do?

Troubleshooting

🎙 **Spoken answer · ~61 s**

I treat it as a possible consumer or broker problem. First I check the cluster status and alarms, memory and disk watermarks, and whether any node is partitioned. Then I list queues sorted by message count and look at consumers per queue: a queue with many messages and zero consumers means a service that should be reading it is down or disconnected, for example a nova-compute whose queue grew while it was offline. I check that service's logs and restart it if it lost its connection. If the growth comes from notification queues with no consumer, I purge them or set a TTL and a length limit. If the broker is memory-starved, I find the heaviest queues and connections. I also check network latency, heartbeat settings and clock skew. After fixing the cause, I watch the queues drain and confirm that API operations are completing again.

Likely follow-ups

- Why do notification queues grow unbounded?
- What does a queue with zero consumers mean?

Check cluster status and alarms, find queues with many messages and no consumers, restart or fix the disconnected service, add TTL or limits to notification queues, and check watermarks and partitions.

```
rabbitmqctl list_queues name messages consumers | sort -k2 -nr | head
```

```
rabbitmqctl list_connections name state | head
```

```
rabbitmqctl set_policy TTL '^notifications\..*' '{"message-ttl":3600000}' --apply-to queues
```

Interview tip: Zero consumers on a full queue points to a disconnected service.

📚 **Official docs**

- RabbitMQ: Quorum queues
- RabbitMQ documentation
- oslo.messaging documentation

certs.eknathalabs.com

Q126 Image upload or boot from image is very slow or fails. What do you check?

Troubleshooting

🎙 **Spoken answer · ~63 s**

I follow the data path. For uploads, I check the Glance API logs and the store back end, which may be Ceph, file or Swift, for errors and latency, plus the network path from the client through the load balancer, including timeouts and request size limits in HAProxy or a reverse proxy. For slow boots, I check whether the compute node has the image cached or has to download it, whether the image is qcow2 on a Ceph back end, which forces a conversion instead of a clone, and the network speed between compute and Glance. With a Ceph image back end and raw images, the boot should be an almost instant copy-on-write clone, so a slow boot indicates a missing format or configuration. I also check Glance workers and concurrency limits, and disk space in staging or cache directories. After fixing it, I test with a known image and measure.

Likely follow-ups

- Why does qcow2 slow down boots on Ceph?
- What proxy limits affect large uploads?

Follow the data path: client and load balancer timeouts, Glance store health, image format (raw for Ceph clones), compute cache and bandwidth, and staging disk space.

```
openstack image show ubuntu -c disk_format -c size
```

```
rbd -p images ls
```

```
grep -i 'timeout' /etc/haproxy/haproxy.cfg
```

Interview tip: Raw format on Ceph is the classic fast-boot fix.

📚 **Official docs**

- Glance documentation

certs.eknathalabs.com

Q127 Cinder volume creation from an image is slow or fails. What is your approach?

Troubleshooting

🎙 Spoken answer · ~58 s

I check where the time or failure occurs. Creating from an image means Cinder downloads the image from Glance and writes it to the volume, unless the back end can clone it. With Ceph and raw images in the same cluster, the clone is nearly instant, so a slow create means a format mismatch or missing configuration, such as glance_api_version or the image not accessible by RBD. Without cloning, the data travels through the cinder-volume host, so network bandwidth and temporary space there matter, and the image conversion cache helps repeated creations. I read the cinder-volume log by request ID, check the scheduler decision, back end capacity and status, and the volume type extra specs. Errors in scheduling usually mean no back end matched or has capacity. I also test the storage array directly to isolate array issues from OpenStack.

Likely follow-ups

- What does the image-volume cache do?
- How does Ceph cloning work with Glance?

Determine whether it is scheduling, image transfer or backend. Ceph clones need raw images in the same cluster; otherwise data flows through cinder-volume. Check logs, capacity and type specs.

```
grep <req-id> /var/log/cinder/cinder-volume.log
```

```
cinder get-pools --detail
```

```
openstack volume show vol1 -c status -c os-vol-host-attr:host
```

Interview tip: Say clones versus copies explains most speed differences.

📚 Official docs

- OpenStack documentation
- Cinder documentation
- Glance documentation

certs.eknathalabs.com

Q128 Instances lose network connectivity after a Neutron agent or OVS restart. Why?

Troubleshooting

🎙 **Spoken answer · ~58 s**

Restarting the agent or Open vSwitch can temporarily clear or rebuild flows on the integration bridge, which interrupts traffic until the agent reprograms them. Known problems include flows not being restored in the right order, stale tap devices, tunnel ports missing, and conntrack state lost so existing connections break. Modern OVS agents use a flow-reconciliation approach to keep the data plane during restarts, and OVN keeps flows in ovn-controller, which also minimises disruption. To diagnose, I check agent logs for errors, verify that br-int and br-tun contain the expected ports and flows with ovs-ofctl dump-flows, and compare them with a healthy host. I check the OVS database connection, any ports with errors, and whether the host's ofport values changed. For future maintenance, I restart agents in a controlled, one-host-at-a-time way, and upgrade OVS with attention to datapath behaviour.

Likely follow-ups

- What changed with OVN regarding restarts?
- How do you compare flows with a healthy host?

Restarts can flush or delay flows and lose conntrack. Check agent logs, br-int and tunnel ports and flows against a healthy host, and restart agents one host at a time. OVN is more restart-tolerant.

```
ovs-ofctl dump-flows br-int | wc -l
```

```
ovs-vsctl list-ports br-int | head
```

```
journalctl -u neutron-openvswitch-agent --since '10 min ago'
```

Interview tip: Compare flow counts with a healthy host as a quick signal.

📚 **Official docs**

- OpenStack Neutron
- OpenStack documentation
- Nova documentation

certs.eknathalabs.com

Q129 Users get random 503 or timeout errors from OpenStack APIs. How do you find the cause?

Troubleshooting

🎙 **Spoken answer · ~60 s**

Random errors suggest an unhealthy backend behind the load balancer or a saturated component. I start at HAProxy: which backends are marked down, and the statistics for response codes and queue times per server. Then I check each API service for worker saturation, in particular slow requests blocking all workers, and read their logs at the matching timestamps. I look at the database, for connection pool exhaustion or slow queries, at RabbitMQ for blocked connections, and at Keystone token validation latency, since every call depends on it. Resource limits such as file descriptors, memory and CPU on controllers also matter. If errors correlate with one controller, I compare configuration and Fernet keys across nodes. I use a request ID from a failing call to follow its path, and I confirm the fix by watching the error rate on the dashboard return to baseline.

Likely follow-ups

- What HAProxy statistics are most useful?
- How do slow DB queries cause 503s?

Check HAProxy backend status and per-server errors, API worker saturation, DB pool and slow queries, RabbitMQ blocks, Keystone latency and controller resources; compare nodes if errors follow one server.

```
echo 'show stat' | socat stdio /var/lib/kolla/haproxy/haproxy.sock | cut -d, -f1,2,18,42
```

```
mysql -e 'SHOW FULL PROCESSLIST'
```

```
ss -s
```

Interview tip: If it follows one controller, compare configs and keys.

📚 **Official docs**

- RFC 9110: HTTP Semantics (status codes, methods)
- OpenStack documentation

certs.eknathalabs.com

Q130 An instance has no network after a hard reboot of its compute node. What happened?

Troubleshooting

🎙 Spoken answer · ~63 s

After a host reboot, instances normally restart automatically only if resume_guests_state_on_host_boot is set, and their network wiring must be rebuilt. I check whether the instance is running, then whether its port is bound and active, and whether the OVS or OVN configuration on the host contains the port. A common cause is that the Neutron agent or ovn-controller started before libvirt or OVS was ready, leaving the tap device unplugged, or that the instance started before the network was wired. Another is network namespaces and routers that need re-creation on the host. Hard rebooting the instance through Nova re-plugs VIFs, so openstack server reboot --hard is often the fix. For prevention, I ensure service ordering at boot, enable guest resume with delays, and test host reboot scenarios in staging. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- What does reboot --hard re-plug?
- How do you order services at boot?

Check the guest state, port binding and OVS or OVN wiring. Startup ordering often leaves taps unplugged; a Nova hard reboot re-plugs VIFs. Fix ordering and test host reboots in staging.

```
openstack server reboot --hard web1
```

```
ovs-vsctl list-ports br-int | grep <tap>
```

```
crudini --get /etc/nova/nova.conf DEFAULT resume_guests_state_on_host_boot
```

Interview tip: Say hard reboot re-plugs VIFs, then fix ordering.

📚 Official docs

- OpenStack Nova

certs.eknathalabs.com

Q131 Security group changes do not seem to apply. How do you verify what is enforced?

Troubleshooting

🎙 **Spoken answer · ~63 s**

I confirm at each layer. In the API, I check that the port has the security group I think it has, since a port may use a different group than the instance's default, and that the rule's direction, protocol, port range and remote prefix are correct, including ethertype for IPv6. Then I check what is enforced on the host: with the OVS native firewall, I look at the OpenFlow rules and conntrack entries for the port; with OVN, I check ACLs in the northbound database; with the iptables hybrid driver, I look at the neutron iptables chains for the port. Existing connections may persist because of connection tracking even after a rule is removed, which makes changes look ineffective until the connection closes. Also port security and allowed address pairs can interfere. I test with a fresh connection and capture traffic to see whether packets are dropped by the filter.

Likely follow-ups

- Why do old connections survive a rule removal?
- What are allowed address pairs?

Verify the port's groups and rule details, then the enforced state on the host (OVS flows and conntrack, OVN ACLs or iptables chains). Conntrack can keep old flows alive; test with a new connection.

```
openstack port show <port> -c security_group_ids
```

```
ovn-nbctl list ACL | grep <port>
```

```
conntrack -L | grep <ip>
```

Interview tip: Conntrack explains many rules that appear not to work.

📚 **Official docs**

- Neutron documentation

certs.eknathalabs.com

Q132 The Nova or Neutron database is slow and API calls take seconds. How do you respond?

Troubleshooting

🎙 **Spoken answer · ~57 s**

I find what the database is doing. I look at the process list and slow query log for long-running or locked queries, check connection counts against the limit, and look at Galera health, flow control and replication lag, since a slow node can throttle the cluster. Typical causes in OpenStack are huge tables of soft-deleted rows, missing indexes after an upgrade, an unbounded query from a monitoring script listing all instances across projects, and too many workers opening connections. Fixes include archiving and purging old data, adding the recommended index, rate-limiting or caching the heavy caller, and tuning pool sizes and InnoDB buffer pool memory. If I/O is the bottleneck, I check the disks and use faster storage for the database. I confirm improvements with latency percentiles, and add alerts for slow queries and connection saturation.

Likely follow-ups

- Why does one slow Galera node slow all?
- How do soft-deleted rows hurt performance?

Check the process list and slow query log, connections and Galera flow control, then address deleted-row bloat, missing indexes, heavy callers and pool sizing, and tune buffer pool and disks.

```
mysql -e 'SHOW FULL PROCESSLIST' | head -30
```

```
mysql -e "SHOW STATUS LIKE 'wsrep_flow_control%'"
```

```
nova-manage db archive_deleted_rows --until-complete
```

Interview tip: Monitoring scripts that list everything are a common hidden culprit.

📚 **Official docs**

- OpenStack Nova
- OpenStack Neutron
- OpenStack documentation

certs.eknathalabs.com

Q133 A hypervisor shows wrong resource usage compared with the real state. How do you fix it?

Troubleshooting

🎙 Spoken answer · ~65 s

When Nova's recorded usage and reality diverge, scheduling goes wrong, with phantom no valid host errors or overcommit. The sources are Nova's own database, Placement allocations and the actual libvirt domains. I compare them: list instances on the host in Nova, the allocations for the host's resource provider, and virsh list. Discrepancies come from instances deleted in the database but still running on the host, orphan allocations after failed migrations or evacuations, or manual changes outside OpenStack. Tools like nova-manage placement heal_allocations and audit help reconcile, and orphan domains are cleaned after careful identification. The nova-compute resource tracker updates periodically, so a restart or forced update can fix temporary drift. I then investigate the root cause, such as a failed evacuation process, and add periodic audits so drift is caught early. I always add that the right answer depends on the workload, so I would ask about scale and failure tolerance before committing to a design.

Likely follow-ups

- What does heal_allocations do?
- How do you find orphan libvirt domains?

Compare Nova instances, Placement allocations and libvirt domains; reconcile with nova-manage placement audit and heal_allocations; clean orphans carefully and add periodic audits.

```
nova-manage placement audit --verbose
```

```
nova-manage placement heal_allocations
```

```
virsh list --all --uuid | sort > /tmp/virsh.txt
```

Interview tip: Drift between Nova, Placement and libvirt is the root of many scheduling mysteries.

📚 Official docs

- OpenStack Nova

certs.eknathalabs.com

Q134 After a controller reboot some services fail to start or stay unhealthy. What is your method?

Troubleshooting

🎙 Spoken answer · ~61 s

I confirm the basics, in the order of dependencies. Is the VIP up and on the right node? Is HAProxy running and are backends healthy? Are Galera and RabbitMQ clustered, and has Galera needed a bootstrap? Many OpenStack services fail to start simply because the database or queue is not available yet, so fixing the foundations first usually fixes the rest. I check container or service status, read the logs for the first error rather than the last, and verify that file systems, time sync and DNS are healthy. A full-cluster restart needs the proper order: database, queue, memcached, then Keystone, then the other services. After the system is up, I run service lists for every project, check for stale agents, and run the canary instance. I then review why the reboot caused trouble, for example missing start ordering or a stuck lock, and update the runbook.

Likely follow-ups

- What is the right start order after a full outage?
- How do you find the first error in a log?

Work in dependency order: VIP, HAProxy, Galera, RabbitMQ, memcached, Keystone, then the rest. Read the first error, then run service lists and the canary, and update the runbook.

```
docker ps --filter status=exited
```

```
kolla-ansible -i multinode mariadb_recovery
```

```
openstack compute service list && openstack network agent list
```

Interview tip: Foundations first: DB and queue before services.

📚 Start here (official docs)

- OpenStack documentation

certs.eknathalabs.com

Q135 How do you run Kubernetes on OpenStack, and which integrations matter?

Integration

🎙 Spoken answer · ~57 s

There are two broad approaches. You can create a cluster with Magnum, the OpenStack container orchestration service, or with Cluster API using the OpenStack provider, or use tools like Kubespray, Rancher or Terraform to deploy on instances. Either way, the integrations that matter are the OpenStack cloud provider and CSI drivers. The cloud controller manager creates Octavia load balancers for Services of type LoadBalancer and manages node lifecycle and routes. The Cinder CSI driver provides persistent volumes from Cinder, with topology awareness for availability zones, and the Manila CSI driver provides ReadWriteMany shares. Keystone authentication can map to Kubernetes users. Network design matters too: pod networking over the tenant network, trunk ports with Kuryr, or a CNI like Calico or Cilium on top. I plan quotas, floating IPs, load balancer limits and node image lifecycle upfront.

Likely follow-ups

- What does the cloud controller manager do?
- Magnum versus Cluster API?

Create clusters with Magnum or Cluster API, then use the OpenStack cloud controller for Octavia load balancers, Cinder CSI for volumes, Manila CSI for shared storage and Keystone auth. Plan quotas and networks.

```
openstack coe cluster create --cluster-template k8s-template --node-count 3 k8s1
```

```
kubectl get storageclass
```

```
kubectl get svc -o wide
```

Interview tip: Name Octavia, Cinder CSI and the cloud controller manager.

📚 Official docs

- OpenStack documentation
- Magnum documentation

certs.eknathalabs.com

Q136 What is Magnum and how does it differ from Cluster API?

Integration

🎙 Spoken answer · ~64 s

Magnum is the OpenStack service that provisions container orchestration clusters, mostly Kubernetes, as a first-class API. You define a cluster template with image, flavors, network, and Kubernetes version, then create a cluster; Magnum uses Heat in the background to create the instances, networks, load balancers, and installs the control plane and workers, and issues certificates through Barbican or its own store. It offers lifecycle operations like resize and upgrade. Cluster API is a Kubernetes-native approach: a management cluster runs controllers that create and reconcile workload clusters declaratively, with providers for many platforms, including OpenStack. It gives GitOps friendly workflows and a uniform experience across clouds. I choose Magnum when I want an OpenStack-native, self-service service for tenants, and Cluster API when I want declarative, multi-cloud cluster management and faster tracking of upstream Kubernetes. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- How does Magnum use Heat?
- When would you choose Cluster API?

Magnum is the OpenStack-native container cluster API built on Heat; Cluster API is a Kubernetes-native declarative approach with a provider per platform. Choose by tenancy model and multi-cloud needs.

```
openstack coe cluster template create --image fedora-coreos --coe kubernetes --flavor
m1.medium --network-driver calico k8s-template
```

```
clusterctl init --infrastructure openstack
```

```
kubectl get clusters
```

Interview tip: Frame it as OpenStack-native service versus Kubernetes-native management.

📚 Official docs

- Cluster API book
- Magnum documentation

certs.eknathalabs.com

Q137 How do you use Ceph with both OpenStack and Kubernetes?

Integration

🎙 Spoken answer · ~56 s

A single Ceph cluster can serve both. For OpenStack it provides RBD for Glance, Cinder and Nova, and RGW for object storage. For Kubernetes, either running on OpenStack or elsewhere, you can use the Ceph CSI driver to provision RBD block volumes and CephFS volumes, or Rook, which runs Ceph inside Kubernetes. I separate pools for each consumer with their own CRUSH rules, quotas, and client keys with minimal capabilities so a Kubernetes credential cannot touch OpenStack pools. Network access from the Kubernetes nodes to the Ceph public network must be planned, with firewalls. A benefit is one operational stack and shared capacity, and the risk is a shared failure domain and noisy neighbours, so I use QoS, monitoring and tiered pools. For large separate platforms, I sometimes prefer separate clusters to limit blast radius.

Likely follow-ups

- Why separate pools and keys?
- Rook versus external Ceph?

One Ceph cluster can back OpenStack through RBD and RGW and Kubernetes through Ceph CSI or Rook, with separate pools, keys and CRUSH rules. Watch shared failure domains and noisy neighbours.

```
ceph auth get-or-create client.k8s mon 'profile rbd' osd 'profile rbd pool=kube'
```

```
ceph osd pool create kube 128
```

```
kubectl get pv
```

Interview tip: Say least-privilege keys per consumer.

📚 Official docs

- Ceph: Block devices and OpenStack
- OpenStack documentation
- Ceph documentation

certs.eknathalabs.com

Q138 How would you migrate workloads from VMware to OpenStack?

Integration

🎙 Spoken answer · ~56 s

I plan it as a migration programme. First discovery: inventory VMs, their sizes, dependencies, operating systems, storage and network requirements, and classify by complexity and criticality. Then design the target: flavors that match the common sizes, networks and security groups equivalent to the existing segments, storage tiers, and a landing zone with quotas and identity. For the move itself, options include converting disk images with qemu-img and uploading to Glance, using a migration tool that does continuous replication and cutover, such as Coriolis, or rebuilding applications through infrastructure as code. Guest changes include installing virtio drivers on Windows, replacing VMware Tools with cloud-init and guest agents, and adjusting IP addressing or keeping it with provider networks. I migrate in waves, starting with low-risk systems, testing each, and keeping rollback until the old environment is retired.

Likely follow-ups

- What guest changes are needed?
- How do you keep IP addresses during cutover?

Discover and classify VMs, design flavors, networks and storage tiers, then convert or replicate disks with virtio drivers and cloud-init, migrate in waves with rollback, and rebuild with IaC where possible.

```
qemu-img convert -f vmdk -O raw app.vmdk app.raw
```

```
openstack image create --disk-format raw --file app.raw app-imported
```

```
virt-v2v -i vmx app.vmx -o openstack
```

Interview tip: Mention virtio drivers for Windows guests.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q139 How do you connect OpenStack to a hybrid environment with a public cloud? Integration

🎙 Spoken answer · ~58 s

I start with connectivity: a site-to-site VPN or dedicated links to AWS, Azure or Google, with BGP for dynamic routing and non-overlapping address ranges from the start, since overlaps are expensive to fix. Neutron VPNaaS or an external router handles IPsec, and a transit hub in the public cloud connects several VPCs. Next identity: federate Keystone and the public cloud with the same IdP so users have a consistent login. Then tooling: use Terraform or Ansible modules for both clouds, with a common tagging policy for cost and ownership, and a single observability stack that scrapes both. For data, I decide what lives where, considering egress cost and latency, and use replication or object storage compatible with S3 through Ceph RGW. For application portability, containers and Kubernetes reduce differences. Governance keeps the hybrid design from becoming two unmanaged silos.

Likely follow-ups

- Why plan non-overlapping CIDRs first?
- How do you handle egress costs?

Non-overlapping addressing, VPN or dedicated links with BGP, shared IdP, common tagging and IaC tooling, unified observability, and a data placement strategy that accounts for egress cost.

```
openstack vpn service create --router r1 vpn1
```

```
terraform apply -var cloud=openstack
```

```
aws ec2 describe-vpn-connections
```

Interview tip: Say address planning comes first.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q140 How do you use the openstacksdk or APIs to build automation, and what pitfalls exist?

Integration

🎙 Spoken answer · ~63 s

openstacksdk gives a clean Python interface across services. You connect using clouds.yaml, then call methods such as conn.compute.create_server or conn.network.create_network, with consistent objects and pagination handled for you. For scripts I follow some rules. Use application credentials with least privilege. Wait for resources with the provided wait helpers instead of sleeping. Make operations idempotent: look up by name or tag first, so reruns do not create duplicates. Handle quota and conflict errors gracefully, retry on transient errors with backoff, and log request IDs for support. Pin microversions where behaviour matters. Pitfalls include listing everything across all projects, which loads the control plane, forgetting pagination or filters, and long sequences without timeouts. For infrastructure, I still prefer Terraform or Ansible for state, and use the SDK for glue logic and tooling. To make it concrete, I would describe a real situation where I applied this and what I measured before and after.

Likely follow-ups

- Why use filters rather than listing everything?
- How do you make a script idempotent?

openstacksdk with clouds.yaml and app credentials, wait helpers, idempotent lookups, backoff on transient errors, logged request IDs and filtered queries. Use IaC for state, SDK for glue.

```
python -c "import openstack; c=openstack.connect(cloud='prod'); print([s.name for s in c.compute.servers(status='ACTIVE')])"
```

```
conn.compute.wait_for_server(server, status='ACTIVE')
```

```
openstack --os-cloud prod server list --limit 100
```

Interview tip: Idempotence and filtered queries are the two habits to mention.

📚 Official docs

- OpenStack documentation
- openstacksdk documentation

certs.eknathalabs.com

Q141 How do you build CI/CD pipelines that deploy to OpenStack?

Integration

🎙 Spoken answer · ~58 s

The pipeline authenticates with an application credential or, better, short-lived credentials from a secret manager, never an admin account, and runs on a runner that can reach the API endpoint. Stages typically include building an image or artifact, with Packer or diskimage-builder if needed, validating infrastructure code with fmt, validate and policy checks, planning with Terraform, applying to a test project, running smoke tests, and promoting the same code to production by changing variables. Each environment is a separate project or even separate cloud, with its own quotas, so a bug cannot damage production. Ephemeral test environments created per branch and destroyed afterwards keep costs under control, and tagging resources with the pipeline run ID allows cleanup of leftovers. State is stored remotely with locking, and approvals gate production applies. Failures produce clear logs, including OpenStack request IDs.

Likely follow-ups

- Why create per-branch environments?
- How do you clean leaked test resources?

Scoped short-lived credentials, build and validate, plan, apply to a test project with smoke tests, promote with variables, ephemeral per-branch environments tagged by run ID for cleanup, remote state and approvals.

```
terraform plan -var project=test -out=plan
```

```
openstack server list --tag run=1234 -f value -c ID | xargs -r openstack server delete
```

```
openstack --os-cloud ci stack delete -y env-$BRANCH
```

Interview tip: Tag everything with the run ID so cleanup is trivial.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q142 What is NFV on OpenStack and which features make it work?

Integration

🎙 **Spoken answer · ~61 s**

Network Function Virtualisation runs telecom network functions, such as firewalls, routers, packet cores and virtual base-station components, as software on cloud infrastructure. They need high throughput and predictable low latency, so OpenStack offers a group of performance features: CPU pinning and isolation, NUMA-aware placement, hugepages, SR-IOV and DPDK-accelerated Open vSwitch for fast packet processing, real-time kernels for strict timing, and trunk ports and multiple networks for complex topologies. Placement and traits ensure functions land on suitable hosts, and host aggregates separate NFV compute from general workloads. Orchestration is through ETSI MANO tools like Tacker or external orchestrators. The trade-offs are less flexibility, harder live migration, more complex capacity planning and the need for tight hardware and firmware tuning. NFV is a major reason large operators adopted OpenStack. If the interviewer pushes further, I would explain how I would test this in a staging environment before touching production.

Likely follow-ups

- What does DPDK change?
- Why are NFV hosts separated by aggregates?

NFV runs network functions as software using CPU pinning, NUMA, hugepages, SR-IOV or DPDK, trunk ports and dedicated aggregates, orchestrated by MANO tools. It trades flexibility for deterministic performance.

```
openstack flavor set vnf --property hw:cpu_policy=dedicated --property hw:mem_page_size=1GB
```

```
dpdk-devbind.py --status
```

```
ovs-vsctl get Open_vSwitch . other_config:dpdk-init
```

Interview tip: List the performance features together; that is the answer.

📚 **Official docs**

- OpenStack documentation

certs.eknathalabs.com

Q143 How do you integrate OpenStack with external monitoring and ticketing systems?

Integration

🎙 Spoken answer · ~57 s

I treat OpenStack as one more source in the wider operational platform. Metrics flow to Prometheus through exporters for OpenStack APIs and node and Ceph exporters, into the same Grafana and Alertmanager that the organisation already uses. Logs go to the central logging platform with request IDs and tenant fields. Alerts route through Alertmanager to the incident tool such as PagerDuty or Opsgenie, with runbook links, and create tickets automatically in the ITSM system for non-urgent issues. In the other direction, tenant requests, onboarding and quota changes flow from the ticketing or portal system into automation via webhooks and APIs, so there is an audit trail. Inventory data from OpenStack feeds the CMDB for asset tracking. Events from the notification bus, such as instance creation, can trigger downstream processes, and I keep these integrations documented and tested.

Likely follow-ups

- How do you avoid alert duplication?
- What goes into a CMDB from OpenStack?

Exporters feed Prometheus and Grafana, logs go to the central stack, Alertmanager routes pages and tickets with runbook links, and requests flow from ITSM into automation. Inventory feeds the CMDB.

```
curl -s localhost:9180/metrics | head
```

```
amtool alert query
```

```
curl -X POST $ITSM/api/tickets -d @ticket.json
```

Interview tip: Say you integrate with existing tools rather than building a parallel stack.

📚 Official docs

- OpenStack documentation
- Swift documentation

certs.eknathalabs.com

Q144 Why might a team choose OpenStack today instead of public cloud, and what are the honest downsides?

Integration

🎙 Spoken answer · ~61 s

Reasons to choose OpenStack include data sovereignty and compliance that require infrastructure inside a specific facility or country, predictable cost at large steady scale where public cloud pricing and egress fees become heavy, performance control such as bare metal, GPUs and NUMA tuning, avoiding vendor lock-in with open APIs, and the need to integrate with existing hardware or telecom needs. It is strongest where the organisation has the scale and the skills to run it. The honest downsides are the operational burden, because you own upgrades, security patching, capacity and on-call, a smaller ecosystem of managed higher-level services compared to the hyperscalers, slower feature delivery for developers, and the risk of complexity if the team is small. The decision is a total cost and capability comparison, and many organisations run a hybrid, using OpenStack for steady and regulated workloads and public cloud for bursty and specialised services.

Likely follow-ups

- At what scale does private cloud become cost effective?
- What skills must the team have?

Choose it for sovereignty, steady large-scale cost, performance control, open APIs and special hardware. Downsides: operational burden and fewer managed services. Often run hybrid with public cloud.

```
openstack hypervisor stats show
```

```
aws pricing get-products --service-code AmazonEC2 --max-items 1
```

```
openstack usage list
```

Interview tip: Give the honest downsides; interviewers value balance.

📚 Official docs

- OpenStack documentation

certs.eknathalabs.com

Q145 How does console access work for instances and how do you troubleshoot it?

Nova

🎙 Spoken answer · ~63 s

Nova offers several console types. The most common is VNC through the noVNC proxy: the user requests a console URL, Nova creates a short-lived token, and the nova-novncproxy service on the controller connects the browser to the VNC server that libvirt exposes on the compute host. SPICE works similarly, and a serial console gives a text-based option through the serial proxy, which is also useful for scripting and for headless guests. For troubleshooting, I check that the proxy service is running and reachable through the load balancer with WebSocket support, that the compute host's novncproxy_base_url and vncserver_proxyclient_address settings are correct and routable, that the token has not expired, and that a firewall is not blocking the VNC port range from the proxy to the compute node. For a broken guest with no console output, I read the console log through the API, which captures the serial output, to see boot errors.

Likely follow-ups

- Why does the load balancer need WebSocket support?
- What does the console log show?

noVNC proxy uses short-lived tokens to reach libvirt's VNC on the compute host. Check proxy reachability, base URL and proxyclient address, WebSocket support and firewall ports; use the console log for boot errors.

```
openstack console url show web1
```

```
openstack console log show web1 --lines 50
```

```
ss -ltnp | grep novncproxy
```

Interview tip: The console log is often faster than the graphical console.

📚 Official docs

- Nova: Remote console access
- Nova documentation

certs.eknathalabs.com

Q146 How does L3 HA with VRRP work for legacy Neutron routers?

Neutron

🎙 Spoken answer · ~58 s

In the classic ML2 OVS design, a legacy router lives on a single network node, which is a single point of failure. L3 HA runs the same router on several network nodes in an active-standby arrangement using keepalived and VRRP. Neutron creates an HA network, a hidden tenant network, per project, so the router instances can exchange VRRP heartbeats. One node holds the master role and owns the router's gateway and floating IP addresses, and if it fails, a standby takes over within a few seconds by sending gratuitous ARP so upstream switches learn the new location. Connections tracked on the old master are lost unless conntrackd is configured to synchronise state. It is simple and reliable but consumes extra networks and namespaces. With OVN, high availability is provided through gateway chassis priorities and BFD, so VRRP is not needed.

Likely follow-ups

- How is the failover detected?
- What replaces VRRP in OVN?

Routers run active-standby on several network nodes, exchanging VRRP heartbeats on a hidden HA network; the backup takes over and sends gratuitous ARP. OVN uses gateway chassis priorities with BFD instead.

```
openstack router set --ha r1
```

```
neutron l3-agent-list-hosting-router r1
```

```
ip netns exec qrouter-<id> ip a | grep -E 'inet |state'
```

Interview tip: Say OVN makes VRRP unnecessary through gateway chassis and BFD.

📚 Official docs

- OpenStack Neutron
- Neutron: L3 HA with VRRP
- OpenStack documentation

certs.eknathalabs.com

Q147 How do Glance multi-store, image import and caching improve image handling?

Storage

🎙 **Spoken answer · ~60 s**

Glance can be configured with several storage back ends at once, for example a local file store, Ceph and S3, and you choose where an image is stored through the store parameter or a default. The image import workflow replaces the old direct upload: you stage the data first, then an import process runs plugins such as format inspection, conversion to raw, and metadata injection, and copies the image to chosen stores. That gives control and safety, for example rejecting malformed images and standardising formats for Ceph. Caching reduces repeated downloads: the Glance cache stores frequently used images locally on the API node, and Nova caches base images on each compute host, so only the first boot per host pays the transfer cost. With Ceph in all roles, cloning makes most of this unnecessary. I also set image quotas and visibility to control growth.

Likely follow-ups

- What can an import plugin do?
- When is the Glance cache useful?

Glance supports multiple stores and a staged import workflow with plugins that validate and convert images, plus API-side and compute-side caches. Ceph cloning largely removes the need for caching.

```
glance image-import --import-method glance-direct --stores ceph <image-id>
```

```
openstack image stores info
```

```
glance-cache-manage list-cached
```

Interview tip: Mention format conversion to raw on import for Ceph.

📚 **Official docs**

- OpenStack documentation
- Glance documentation

certs.eknathalabs.com

Q148 How do you use Tempest and Rally to test an OpenStack cloud?

Core

🎙 Spoken answer · ~58 s

They answer different questions. Tempest is the functional and integration test suite: it exercises the APIs of each service with thousands of tests, which confirms that the cloud behaves according to the API contract. I run a smoke subset after every deployment or upgrade, and a larger selection before accepting a release, then fix or explain failures. Rally is for benchmarking and load: it runs scenarios such as boot and delete servers at a given concurrency, measures durations and failure rates, and shows how the cloud behaves under stress, which helps find bottlenecks and validate capacity changes. I use both in staging and sometimes in production with a limited, tagged project, and store results over time to detect regressions. Combined with a continuous synthetic canary, they give confidence that the cloud works and that it keeps working under growth.

Likely follow-ups

- Which would you run after an upgrade?
- What do Rally results guide?

Tempest verifies API functionality and compatibility, with a smoke subset after each change. Rally measures performance under load. Use both, and keep results to spot regressions.

```
tempest run --smoke
```

```
rally task start boot-and-delete.json
```

```
rally task report --out report.html
```

Interview tip: Tempest for correctness, Rally for performance.

📚 Official docs

- OpenStack documentation
- Tempest documentation

certs.eknathalabs.com

Q149 What is fencing (STONITH) and why does a clustered control plane need it?

HA & scale

🎙 **Spoken answer · ~59 s**

Fencing is the act of forcefully isolating or powering off a node that is presumed failed, so that it can no longer touch shared resources. It is called STONITH, shoot the other node in the head. In a cluster managed by Pacemaker, if a node stops responding, the cluster cannot be sure it is truly dead, it might just be partitioned or hung and about to resume writing to shared storage. Without fencing, two nodes could think they are the active one, which is split brain and risks data corruption. Fencing uses out-of-band mechanisms, such as IPMI or Redfish power control, or a network power switch, or storage-based fencing, to guarantee the node is off before resources move. Instance HA with evacuation relies on fencing for the same reason: you must be certain the old host cannot keep running the guest.

Likely follow-ups

- What happens to evacuation without fencing?
- Which out-of-band tools perform fencing?

Fencing powers off or isolates a presumed-failed node via IPMI, Redfish or a power switch before resources move, preventing split brain and corruption. Evacuation needs the same guarantee.

```
pcs stonith status
```

```
pcs stonith fence node2
```

```
ipmitool -I lanplus -H bmc2 -U admin chassis power status
```

Interview tip: Say you must be certain the old node is off before recovering.

📚 **Start here (official docs)**

- OpenStack documentation

certs.eknathalabs.com

Q150 What are unified limits and how do they change quota management?

Operations

🎙 Spoken answer · ~58 s

Traditionally each service stored quotas in its own database and applied them per project, which made it hard to manage consistently across Nova, Cinder and others. Unified limits move quota definitions into Keystone as the single source of truth. Operators register limits per service and resource, as registered limits that serve as defaults, and then create project-specific limits as overrides. The services, through the oslo.limit library, enforce them and report usage. Keystone also supports hierarchical projects, so a parent project can have limits that bound the sum of its children, which suits departments with subteams. The benefits are consistent tooling and one place to audit limits, and the ability to delegate. The caveats are that adoption differs by service and release, so I check which services use them, and I test the migration of existing quotas carefully before enabling.

Likely follow-ups

- What is a registered limit?
- How do hierarchical limits help departments?

Unified limits store quota definitions in Keystone as registered defaults and per-project overrides, enforced by services through oslo.limit, with hierarchical projects. Check per-service adoption before migrating.

```
openstack registered limit create --service nova --default-limit 20 --region RegionOne servers
```

```
openstack limit create --service nova --project payments --resource-limit 50 --region RegionOne servers
```

```
openstack limit list
```

Interview tip: Say Keystone becomes the single source of truth for limits.

📚 Official docs

- Nova: Quotas
- Keystone: Unified limits

certs.eknathalabs.com